


# ARCHON AGENTIC OS 2026 — 完全機能リスト

## システム・モジュールカタログ、アーキテクチャ概要および機能マトリクス

状況： 11. 11 月 2026 | バージョン： v0.5.1 („Archon Hardened“) | 言語： 日本語

システム・モジュールカタログ、アーキテクチャ概要および機能マトリクス

© Michael Keller

**目次**

1. バージョン状況とリリース概要 (v0.5.1) ..... 3
2. システムビューと専門スタジオ（UI ビュー） ..... 3
3. トークン節約・FinOps エコシステム（コスト最小化） ..... 8
4. カーネル、スケジューリングおよびデーモンアーキテクチャ（柱 1） ..... 11
5. マルチエージェントプロセスモデル、階層および統合（柱 2） ..... 13
6. MCP ドライバー、外部サービスおよびマルチモーダルツール（柱 3） ..... 14
7. 記憶、データベース衛生および Obsidian セカンドブレイン（柱 4） ..... 16
8. セキュリティ、ゼロトラストガバナンスおよびシステム診断（柱 6） ..... 18
9. 自己進化、マーケットプレイスおよび自動文書化（柱 7） ..... 19
10. 参照マトリクス：制御可能な Evey/Hermes プラグイン ..... 20
11. エンタープライズ・レジリエンスアーキテクチャ ..... 22
12. 自律性・サンドボックス・ガバナンスアーキテクチャ ..... 23
13. パフォーマンスコア、スループットスケーリングおよびトークン効率 ..... 25
14. 最新式メモリ・RAG アーキテクチャ ..... 26
15. 信頼性の高い実行と決定性（耐久実行） ..... 28
16. セキュリティ、データ保護およびエンタープライズガードレール ..... 29
17. 自己最適化、評価および群知能 ..... 30
18. 柱 3：経済・ERP（アルファ）およびテナントスイート ..... 32
19. アーキテクチャ・セキュリティ・デプロイメント機能 (v0.5.0 – v0.5.1) ..... 32
20. 五十音順用語集・索引 ..... 33

# 1. バージョン状況とリリース概要 (v0.5.1)

| 版 | Codename | Wesentliche アーキテクチャ- & Feature-標石 |
|---|---|---|
| **v0.5.1** | Archon Hardened | Modulare FastAPI-Router (main.py -88%), 25 kritische + 68 hohe Security-Befunde behoben, Docker-Admin-Proxy Klasse A, RBAC-Benutzerverwaltung (PBKDF2), 2-段階 システム更新 (?local_only=true), AnythingLLM Proxy-統合, portabler 1-Click-Installer. |
| **v0.5.0** | 経済・ERP (alpha) | Vollständige 柱 3 (CRM/PocketBase-Sync, 品目/役務, 案件 ANG/AB, 会計 と 共に DATEV & GiroCode, 法人-テナント), Research Lab v2 & 55 Boardroom-Frameworks. |
| **v0.3.x** | Sentinel / Foundation | 7-柱 OS-Kernel, 70 専門家-Kader, 34+ Evey-差込, FinOps 3-Tier-Routing, 暫定的な 符号砂場, 再演 Debugging, PII-Scrubber, Semantic Cache & FTS5 Hybrid Search. |

# 2. システムビューと専門スタジオ（UI ビュー）

**1. 観覧: Mission Control (Monitoring-Cockpit & Leitstand)**

**問題:** Ohne zentrales Echtzeit-Cockpit fehlen の 概要 について aktive エージェント, Token-吐出量, Tageskosten と blockierte 承認.

**Mechanik:** Bündelt 5 vitale KPI-Kacheln (Flottenlast, 吐出量 TPM, Kostenwächter, 安定性, Approval Gate), 5-列-Board-Projektion, Active Process Inspection Bar (3-Slot-Footer のために Heartbeat, Tool-Concurrency と Context Saturation), 15-Minuten-Token-Aktivitätsband (SVG-Sparkline) と 実況-監査-Stream.

***Nutzen:*** *Liefert 一つの verzögerungsfreie 360°-監視 の gesamten Betriebssystems auf 一つの einzigen Bildschirm.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**2. 観覧: 運用 – Kanban Board & Task-Engine**

**問題:** 任務 in reinen Chat-Verläufen gehen の下に と lassen sich weder の後に 状態 まだ の後に Verantwortlichkeiten 制御する.

**Mechanik:** 5 Lebenszyklus-列 (To-Do, 処理中, Review & QA, Blockiert, 完了) direkt auf kanban.db (Single Source of Truth). Unterstützt konfigurierbare WIP-Limits pro 列, Prioritätsfilter (P0–P3), エージェント-Zuweisung, Zeiterfassung と 1-Klick-Obsidian-Verknüpfung. Integriert 四眼 品質関門: Vor Verschieben 一つの 任務 auf 'done' prüft 一つの Peer-Reviewer (Katharina Völk のために QM/遵守 又は Robert) Arbeitsergebnis auf Vollständigkeit, Substanz と Handlungsfähigkeit (1–100% Score). Bei 'changes_requested' greift 一つの 状態-Sperre; 一つの 人間参加型 Override (force=True) übersteuert に際して Bedarf.

***Nutzen:*** *Strukturiert 全ての operativen 企画 透明な と verhindert Systemüberlastung を通じて Work-in-Progress-Schranken.* Verhindert unfertige Arbeitsstände im 'done'-状態 を通じて zwingendes 四-Augen-Review.

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**3. 観覧: 運用 – Interaktives 2D-Organigramm & Berichtslinien**

**問題:** Flache 一覧 の エージェント zeigen 無 Weisungsbefugnisse 又は Eskalationspfade に際して blockierten Prozessen.

**Mechanik:** Visualisiert 56 Knoten と 55 Berichtslinien als interaktiven 2D-Graphen (Apex -> CEO Robert -> Fach-Leads -> 専門家). Unterstützt Drag & Drop への Umhängen の Vorgesetzten, Raster-Ausrichtung, 班-

Karten-観覧 と dauerhafte 蓄積 in config/org_hierarchy.json.

***Nutzen:*** *Definiert verbindliche Eskalations- と Delegationsketten, denen システム に際して Blockaden 自動 の folgt.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**4. 観覧: 運用 – 内部 エージェント-伝達 (Funkverkehr)**

**問題:** Wenn 専門家 untereinander Teilaufgaben 引き渡す, überflutet dies entweder 利用者-Chat 又は bleibt unsichtbar.

**Mechanik:** Dedizierte Abhörzentrale と共に Kanalliste (#一般の, #tech-dev, #legal, #marketing) と chronologischem 通知-Feed aller Inter-エージェント-Absprachen aus kanban.db.

***Nutzen:*** *Macht 協働 の エージェント-Flotte nachvollziehbar, 無しに primäre 利用者-Postfach zu belasten.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**5. 観覧: Eingang & Triage (3-列 HITL-Gatekeeper & Postfach)**

**問題:** Unkontrollierte Schreibaktionen の エージェント (z. B. 電子郵便-発送) so どのように verstreute Eingänge aus Mail と Telegram bergen Sicherheitsrisiken.

**Mechanik:** Dreispaltige Triage-Zentrale: 列 1 のために gelb markierte 人間参加型-承認 (公開/拒否する), 列 2 のために IMAP/SMTP-電子郵便-Synchronisation と共に Auto-Triage & 委任, 列 3 のために gespiegelten 実況-Telegram-Stream (@Beatman_bot).

***Nutzen:*** *Stellt sicher, dass 無 外部の Schreiboperation 無しに ausdrückliche 承認 の Betreibers ausgeführt wird.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

6. 観覧: エージェント部隊 & Flottenmanagement (8-Reiter-Cockpit)
問題: 専門家 benötigen individuelle Modelle, システム-Prompts, 道具 と Arbeitsanweisungen an 一つの zentralen Ort.
Mechanik: Verwaltet 70 専門家-Profile と共に Schnellfiltern (有効, Favoriten, 全て, 7 Domänen), Hot-Swap のために 提供者 & 模型 so どのように 8 Fach-Reitern: 概要 (360°-Dossier & Telegram-Rufname), Direkt-Chat, 能力 (Skills), 運用 (Task-Historie), SOUL (システム-プロンプト-Editor), 人事記録 (SOPs), Talent-市場 と Skill-市場.
Nutzen: Zentrale Personalabteilung への sekundenschnellen 較正 と 拡張 の gesamten エージェント-Stamms.
価格 & Schattenseite (Trade-off): Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**7. 観覧: Master Chat (Dual-Mode Workspace & Multi-Session-Sidebar)**

**問題:** Beim direkten Arbeiten と共に エージェント fehlte 一つの fokussierte Vollbild-Umgebung と共に schnellem Wechsel の間に 1:1-Dialog, Gruppenchat と Missions-注文.

**Mechanik:** Bietet Dual-Mode-Switch ('Direkt' のために sofortige 応答 vs. 'Mission' のために strukturierte Task-添付), Multi-File-Upload (bis 5 綴), Browser-Screen-Capture (Kamera-Button), Spracheingabe (Diktat), Reasoning-予算-Wähler (0 bis 16k Tokens), KI-プロンプト-Optimizer と rechte Session-Historie (エージェント vs. 群). Beinhaltet Telegram Voice-Pipeline: 受信の Sprachnachrichten (.ogg/Opus) の unterwegs となる via Whisper/Gemini transkribiert, に基づき の 呼称 (/robert, /anwalt etc.) an Fachagenten geroutet と

Antworten via edge-tts (de-DE-ConradNeural) als natürliche Sprachmemos direkt auf Telegram zurückgesendet.

***Nutzen:*** *Vereint Hochgeschwindigkeits-Dialog, マルチモーダルの 入力 と Missions-制御 in 一つの durchgängigen Arbeitsfläche.* Ermöglicht 完全な mobile Sprachbedienung と bidirektionalen Audio-Austausch について Telegram.

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**8. 観覧: AI Mastermind & Boardroom (55 方法論-フレームワーク）**

問題: Komplexe 戦略的な 決定 erfordern strukturierte Management-Methoden と kontroverse Perspektiven statt Einzelmeinungen.

Mechanik: Simuliert mehrstufige Gremiensitzungen (Chairman + 3 reale Fachagenten-Slots + Red-班-Challenger) に基づき の 34 auswählbaren Frameworks in 7 Kategorien (35 Base) (z. B. CLEAR, 意匠 Sprint, Double Diamond, 簡素 Startup, First Principles, Game Theory, Blue Ocean, 事前葬儀, Six Sigma).

Nutzen: Erzwingt methodische Strenge, deckt blinde Flecken を通じて Red-班 auf と liefert entscheidungsreife Board-Memos.

価格 & Schattenseite (Trade-off): Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**9. 観覧: Showroom (企画-Explorer, Artefakt-Galerie & Python-Sandbox)**

**問題:** Generierte 報告, HTML-Prototypen, Kalkulationen と Skripte gehen in langen Konversationen verloren.

**Mechanik:** Katalogisiert 全ての Arbeitsergebnisse の後に Ersteller, 日付 と Typ. Beinhaltet Workspace Vault Reader と共に 実況-Rendering のために Markdown/HTML/JSON so どのように 一つの isolierte Python-Sandbox への sicheren Code-実行. 高度 um interaktiven Showroom 実況-走者: Ermöglicht direkte 実行 の Python- と Code-綴 im Browser と共に 12s Timeout, isoliertem Scratch-目録 と ausklappbarem Terminal-引出 (STDOUT, STDERR, Exit-Code と Millisekunden-存続期間).

***Nutzen:*** *Dient als 受領- と Präsentationshalle のために 全ての 製造する Deliverables の前に 顧客- 又は Produktionseinsatz.* Code-Artefakte と Skripte lassen sich 無しに Konsolenwechsel direkt im Showroom verifizieren と testen.

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

10. 観覧: 中央図書館 & 2nd Brain (RAG, 自動 OCR & Obsidian-Vault)

問題: Fachbücher, DIN-規格 と Bebauungspläne als PDF konnten の エージェント bisher ない 無しに manuelles Kopieren durchsucht werden.

Mechanik: Vereint 3 Sub-Tabs と 4 Core Hubs (00_HR_Agenten, 02_Wissensbibliothek, 記録/, 00_Notebook_Synthesen): 1. AnythingLLM RAG-索引 (Port 3001), 2. Dokumenten-収集者 と共に Drag & Drop, 自動 OCR (Docling) と KI-Themenklassifikation, 3. Direkter Obsidian-Dateibaum & Markdown-Viewer inklusive Vault-Umschalter.

Nutzen: Verwandelt PDF-鑑定書 vollautomatisch in durchsuchbares Markdown- と Vektor-知識 のために 全ての 70 エージェント.

価格 & Schattenseite (Trade-off): Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**11. 観覧: Workflow-Engine (n8n Automation Mesh auf Port 5678)**

**問題:** Wiederkehrende Schnittstellenprozesse (Cron-Jobs, Webhooks, データベース-Abgleiche) verbrauchen

unnötig Tokens, wenn sie について LLMs laufen.

**Mechanik:** Betten visuellen n8n-Workflow-Editor direkt 一つの と verbindet ihn について Skill n8n-automation-mesh (FastMCP) と共に CEO-エージェント Robert. Aktivierte 0-Token-Routinen: 1. Morning-Briefing (werktags 07:30 Uhr と共に Obsidian-Ablage と Telegram-Zustellung), 2. Weekly Maintenance (sonntags 23:00 Uhr と共に SQLite-Backup & Health-監査), 3. RSS & News 監視役 (stündlicher Scan), plus 5 aktive Funktions-Webhooks のために Web-Extraktion, Messaging, Dokumenten-Rendering, Feed-検索 と CRM-Sync.

***Nutzen:*** *Kombiniert deterministische 0-Token-Hintergrundprozesse と共に kognitiven エージェント-決定 an Schlüsselstellen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

### 12. 観覧: LLM 門 (OmniRoute AI Router 計器盤)

**問題:** feste Bindung an 一つの einzigen Cloud-Anbieter führt に際して Rate-Limits 又は Ausfällen への sofortigen Systemstillstand.

**Mechanik:** Zentrales Proxy-計器盤 (Port 20128 / 8000) と共に Multi-提供者-Vault, Echtzeit-Latenzmessung (ms), HTTP-Fehlermonitor (429/500), automatischem Circuit-Breaker と 4-stufiger Fallback-Kaskade (GLM-4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> lokales Llama 3.3 70B).

***Nutzen:*** *Sichert 99,9 % 可用性 aller kognitiven 過程 に際して minimalen API-費用.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

### 13. 観覧: Token Guard (Guardian & Token 分析 in 3 Sub-Views)

**問題:** Fehlende 透明性 darüber, welcher Agent, welches 模型 又は welche Einzelsitzung meisten Tokens と 費用 verursacht.

**Mechanik:** Bietet Zeitfilter (24h, 7d, 30d, 全体), Inaktivitäts-Timer と 3 Sub-Views: 1. 概要 & Tages-傾向-Chart と共に Profil-/模型-表 と Schutz-Schranken-書式, 2. Einzel-Session Verbrauchs-Inspektor (Input, Output, Cache-Read Tokens sortierbar), 3. 資源 & Task-抑制 (Schieberegler 1–16 Tasks).

***Nutzen:*** *Volle Kostenkontrolle bis auf einzelne Session-ID herab inklusive sofortiger 予算- と Parallelitäts-Begrenzung.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

### 14. 観覧: 拡張 & モジュール-Zentrale (34 Evey-Spezial-差込)

**問題:** 安全, 記憶- と Analysefunktionen ließen sich bisher ない modular pro Fachgebiet zu- 又は abschalten.

**Mechanik:** Steuert 34 spezialisierte Hermes/Evey-差込 について 8 Fach-Reiter: セキュリティ & Wächter (4), Performance & Cache (2), Monitoring & 分析 (3), エージェント-Intelligenz (6), Planung & Tasks (2), Brücken & 接続口 (4), 記憶 & 知識 (3), Tools & Execution (3) plus 財務/品質/保守. Beinhaltet standardisierte Pluginsystem & Runtime-Hooks: Modulare Ordnerstruktur (plugins/<plugin_id>/plugin.json), 4 Lifecycle-Hooks (on_load, on_enable, on_disable, on_unload) と 2 Tool-Interzeptoren (before_tool_execute, after_tool_execute) と共に vorinstallierten Starter-差込 (システム Metrics, Security 監査 Hook, Cost Guard).

***Nutzen:*** *Erlaubt 精密な Zu- と Abschalten einzelner Systemfähigkeiten im laufenden 運用 経由 Mausklick.* Ermöglicht isolierte Runtime-拡張 と globale Tool-Interzeption 無しに Eingriff in Kernel-Code.

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**15. 観覧: Research Lab (Deep Search & Evidenz-Triangulation)**

**問題:** Einfache Suchanfragen 納品する ungeprüfte Einzeltreffer 無しに Quellenabgleich 又は Vertrauensbewertung.

**Mechanik:** Mehrstufiges Forschungs-Cockpit と共に wählbarer Suchtiefe, 情報源-Monitor, Evidenz-Score, 実況-統合-記録 への Triangulation widersprüchlicher Aussagen と 1-Klick-搬出 in Obsidian-Vault. Beinhaltet 自律的な Tiefenrecherche & Reflexionsschleife v2 (Re-Search Loop): Zerlegung komplexer Recherchen in 3–5 Teilfragen, autonome Lückenanalyse, bis zu 3 iterative Nachfolge-Suchläufe, Multi-提供者 Fallback (SearXNG / Jina Reader) と SSRF-Schutz.

***Nutzen:*** *Erzeugt belastbare, zitierfähige Markt-, Rechts- と 技術-記録綴.* Schließt Wissenslücken selbstständig を通じて rekursive Folgerecherchen に際して widersprüchlichen 又は unvollständigen 情報源.

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**16. 観覧: 遠隔測定 & 記録 (3-情報源 実況-監査 Stream)**

**問題:** Bei Fehlern in Containern 又は エージェント-Prompts musste bisher mühsam im Terminal について Docker-Befehle gesucht werden.

**Mechanik:** Bündelt 3 Log-情報源 in Echtzeit: Physische Docker-容器-記録 (stdout/stderr の 11 容器), kognitive エージェント-記録 と kryptografischen 監査-Trail. Mit Level-濾過 (DEBUG, INFO, WARN, ERROR), Auto-Scroll-Lock と 搬出.

***Nutzen:*** *Unbestechlicher Flugschreiber のために sekundenschnelle Fehlerdiagnose と 遵守-Nachweise.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

17. Spezial-工房: Archon プロンプト 工房 (832 雛形 & Fabric SOP)

問題: 不精密 formulierte 任務 führen zu Halluzinationen, falschen Ausgabeformaten と teuren Korrekturschleifen.

Mechanik: Integrierter Baukasten と共に 824 Enterprise-雛形 in 16 Fachkategorien, について 255 Fabric-SOP-Mustern と の 5-欄-Formel (1. 役割, 2. Kontext, 3. 任務, 4. Format, 5. Constraints/Negativ-規則) so どのように KI-プロンプト-Optimizer.

Nutzen: Zwingt 各 委任 in 一つの glasklare Struktur と überträgt sie 経由 Klick in Chat 又は Kanban-任務.

価格 & Schattenseite (Trade-off): Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**18. Spezial-工房: 視覚の CSS 実況 工房 & 現場の Transform Gizmo (旧 + S)**

**問題:** 意匠-Anpassungen erforderten bisher Suchen in CSS-綴 と ständiges Neuladen の Browsers.

**Mechanik:** Schwebendes Echtzeit-意匠-工房 (旧 + S) と共に Fadenkreuz-DOM-Inspektor, interaktivem Transform-Gizmo (Position ziehen, 4 Ecken/Kanten 規模化する, Eckenrundung ziehen), Theme-Profil-Manager と direktem CSS-搬出 の後に custom_studio.css.

***Nutzen:*** *Ermöglicht pixelgenaues 実況-Styling の gesamten 利用者表面 im laufenden 運用 無しに 頁-Reload.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**19. Spezial-工房: 意匠 システム Lab, AppShell Architekt & 15-Komponenten-工房**

**問題:** Abweichende Header-Höhen, Abstände 又は Kontraste の間に Hell- と Dunkelmodus stören visuelle Ruhe.

**Mechanik:** Beinhaltet interaktiven AppShell-Architekten (56px Header-Flucht, Sidebar-Breite, Viewport-Padding と共に 実況-Miniatursimulation) so どのように 15-Komponenten-意匠-工房 (os-workspace bis os-badge) と共に Dual-Theme-Engine (Light/Dark).

***Nutzen:*** *Garantiert 一つの einheitliches, blendfreies B2B-SaaS-Erscheinungsbild について 全ての 20 観覧 hinweg.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

20. Spezial-観覧: System Control Center (11 Fachreiter, 実況-Dock) & Globale Omnibox (Strg + K)
問題: 設定 waren verstreut; Navigation の間に 20 観覧 と 70 エージェント erforderte viele Mausklicks.
Mechanik: System Control Center bündelt 7 Administrations-Reiter (Hermes Core 含む YAML-Editor, 容器 & サービス, 提供者 & 門, 方法論, 差込, 伝達, 意匠) と lässt sich 経由 '実況-Dock' rechts andocken. Omnibox (Strg + K) navigiert 経由 Tastatur sofort zu jedem Tab (#), エージェント (@) 又は Befehl. Integriert 1-Klick Updatesystem & Continuous Deployment: Automatischer GitHub-Sync と共に Pre-更新 Snapshots aller SQLite-データベース (WAL) in data/snapshots/ の前に jedem Git-Pull, 1-Klick Rollback と minimalen Git-Stat-Prüfungen に対して Mount-Timeouts.
Nutzen: Maximale Bediengeschwindigkeit のために Administratoren と Power-User. Ermöglicht gefahrlose ソフトウェア-Updates と共に automatischem データベース-Rollback im Fehlerfall.
価格 & Schattenseite (Trade-off): Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

## 3. トークン節約・FinOps エコシステム（コスト最小化）

**21. 3-Tier Automatische 費用-Routing-Caps (LOW / MEDIUM / HIGH)**

**問題:** Wenn einfache Routineaufgaben auf teuren Spitzenmodellen (どのように Claude Sonnet 又は GPT-4o) laufen, explodieren Tageskosten.

**Mechanik:** Konfigurierbare 価格-Obergrenzen pro 1 Million Tokens im System Control Center: LOW (max. $0.10/1M -> erzwingt Free-Tier/Flash-Lite), MEDIUM (max. $1.00/1M -> Haiku/Gemini Flash), HIGH (No Cap -> beste Reasoning-Modelle のために komplexe アーキテクチャ).

***Nutzen:*** *Senkt laufenden API-Ausgaben im Alltag um bis zu 85 %, 無しに に際して schwierigen 任務 an 品質 einzubüßen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**22. FreeLLM-API Docker-Proxy & 'auto/best-free' Routing (Port 3002)**

**問題:** Kostenlose Kontingente verschiedener Anbieter は schwer 手動の zu 監視する と wechseln häufig.

**Mechanik:** Ein dedizierter Docker-容器 (freellmapi auf Port 3002) bündelt kostenfreie LLM-Endpunkte. Routing-Profil 'auto/best-free' wählt 自動の stärkste verfügbare Null-欧貨-模型.

***Nutzen:*** *Ermöglicht Dauerbetrieb の Hintergrund- と Triage-エージェント zu exakt $0.00 API-費用.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**23. Stufenlose Reasoning-予算-制御 im Master Chat & Router (0 bis 16k Tokens)**

**問題:** Reasoning-Modelle (Thinking-Mode) verbrauchen oft tausende unsichtbare Denk-Tokens のために triviale

Fragen.

**Mechanik:** 6-stufiger Regler (Aus: 0 Tok, Minimal: ~1k, 低: ~2k, 中: ~4k, 高: ~8k, Maximal: ~16k) im Chat-Footer と in agent_router.py.

***Nutzen:*** *Schaltet teure Denk-Schleifen に際して einfachen 任務完全ab と reserviert tiefes Reasoning のために echte Komplexfälle.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**24. Evey-Cache & Cache-TTL-制御 (24h Semantischer & Exakter 応答-Cache)**

**問題:** Wiederkehrende identische 又は sehr ähnliche Systemabfragen führten 各 Mal zu neuen, kostenpflichtigen API-Calls.

**Mechanik:** Plugin evey-cache speichert Modellantworten と共に konfigurierbarer Lebensdauer (標準: 24 Stunden TTL) と weist im Session-Inspektor eingesparten Cache-Read-Tokens separat aus.

***Nutzen:*** *Beantwortet wiederholte Abfragen in 0 Millisekunden zu 0 Token-費用.*

**価格 & Schattenseite (Trade-off):** は Cache-TTL zu 高い (> 72h), liefert システム に際して dynamischen Themen veraltete Antworten.

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して Echtzeit-Abfragen (実況-Kurse, Serverstatus). Std-TTL: 24h.

**25. Max-Context & Max-History Hard-Clipper im Guardian (32k / 20 Turns)**

**問題:** Endlos wachsende Chat-会議 送信する に際して 各 neuen 問い gesamte Historie erneut と共に と treiben Input-Tokens exponentiell の後に oben.

**Mechanik:** Im Guardian einstellbare Hardschranken のために maximale Kontextfenster (z. B. 32.000 Tokens) と maximale Anzahl übermittelter Chat-通知 (z. B. 最終の 20 Turns).

***Nutzen:*** *Kapselt 最大möglichen Input-費用 pro Turn verlässlich の後に oben ab.*

**価格 & Schattenseite (Trade-off):** Schneidet 通知 oberhalb の 32k Tokens 又は 20 Turns hart ab, wodurch sehr früher Kontext wegfällt.

**Wann ausschalten? & Schwellenwerte:** Für tiefe Dokumenten-Analysen auf 64k–128k im Hermes Core anheben.

**26. Dynamischer Token-Clipper のために Low-Context-Modelle (backend/token_clipper.py)**

**問題:** Kompakte 又は 地域の Modelle (8k/16k) brechen ab, wenn große Tool-Ausgaben Kontextfenster sprengen.

**Mechanik:** Bewahrt zwingend システム-プロンプト (索引 0) と 最終の Benutzeranweisung, kürzt überlange Einzelblöcke intelligent と entfernt ältere Zwischenschritte.

***Nutzen:*** *Erlaubt stabilen Einsatz extrem günstiger Kleinmodelle も に際して umfangreichen 任務.*

**価格 & Schattenseite (Trade-off):** Kürzt überlange Tool-Ausgaben in の Mitte heraus ([...SNIP...]), wodurch einzelne Code-行 im Rohtext fehlen können.

**Wann ausschalten? & Schwellenwerte:** Kurzzeitig 無効化する, wenn 完全 e Roh-記録 ungekürzt benötigt werden.

**27. Sliding Window Memory Compression (Konsolidierter Entscheidungskontext)**

**問題:** Beim einfachen Abschneiden alter 通知 gehen frühere Projektbeschlüsse verloren.

**Mechanik:** Verdichtet ältere Gesprächsrunden zu 一つの kompakten 要約 と injiziert この als Block [KONSOLIDIERTER GESPRÄCHS- & ENTSCHEIDUNGSKONTEXT] auf Position 1.

***Nutzen:*** *Spart bis zu 70 % historische Konversations-Tokens に際して vollem Erhalt aller getroffenen 決定.*

**価格 & Schattenseite (Trade-off):** Komprimierungs-段階 kostet einmalig ~800 Tokens と glättet wörtliche Formulierungen früher Runden.

**Wann ausschalten? & Schwellenwerte:** Bei Bedarf について protect_last_n (Std: 20) im Hermes Core feinjustieren.

**28. LRU-K Page Swapping のために Subagenten-RAM (backend/context_swapper.py)**

**問題:** Parallele Subagenten halten riesige Mengen an Rohdaten gleichzeitig im aktiven プロンプト-記憶.

**Mechanik:** Lagert ない mehr akut benötigte Kontext-頁 ab 80 % 稼働率 の後に Least-Recently-Used-Prinzip in SQLite aus と lädt sie のみ に際して Bedarf (Re-Hydration) zurück.

***Nutzen:*** *Hält Prompts schlank と verhindert unnötige Token-Gebühren のために inaktives Hintergrundwissen.*

**価格 & Schattenseite (Trade-off):** Auslagern と Nachladen の エージェント-RAM auf Disk erzeugt ~10–25 ms I/O-遅延 に際して Kontextwechsel.

**Wann ausschalten? & Schwellenwerte:** Ab 80 % RAM-稼働率 有効な lassen, um OOM-Abstürze zu 防ぐ.

**29. JEV Router — 0-Token Deterministische Intent-Vorfilterung (jev_router.py)**

**問題:** 各 einfache Systemanweisung について 一つの großes Sprachmodell zu klassifizieren, verschwendet Tokens と 時間.

**Mechanik:** Klassifiziert eingehende 入力 の前に LLM-Aufruf 経由 deterministischem JSON Execution Vector と Keyword-Heuristik.

***Nutzen:*** *Leitet klare Systembefehle direkt an Zielroutine weiter, 無しに 一つの einziges LLM-Token zu verbrauchen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**30. ASD-STE100 Simplified Technical English Prompter (.agents/skills/asd-ste100-prompter)**

**問題:** Weitschweifige Prosatexte in システム-Prompts と エージェント-Anweisungen blähen 各 einzelnen API-Call unnötig auf.

**Mechanik:** Transformiert Instruktionen の後に Luftfahrt-標準 ASD-STE100 (eine Bedeutung pro 語, aktive Verben, kurze 文, null Füllwörter).

***Nutzen:*** *Reduziert Token-Länge の システム-Prompts um ca. 30–40 % と senkt gleichzeitig Fehlerrate に際して Tool-Calls.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**31. Anti-AI-Slop & Output-Komprimierung (.agents/skills/anti-ai-slop)**

**問題:** Sprachmodelle erzeugen standardmäßig lange Höflichkeitsfloskeln, Einleitungen と redundante Zusammenfassungen (teure Output-Tokens).

**Mechanik:** Unterbindet KI-Floskeln, leere Phrasen と künstliches Aufblähen direkt im システム-プロンプト と bereinigt Ausgaben.

***Nutzen:*** *Spart bis zu 25 % の besonders teuren Output-Tokens と liefert sofort verwertbare Fachtexte.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**32. Pre-Flight 費用-Schätzer & 予算-予測 (backend/cost_estimator.py)**

**問題:** Vor Start großer 任務 war ない absehbar, was の Lauf kosten wird.

**Mechanik:** Berechnet vorab について POST /api/router/estimate-cost erwarteten Input-/Output-Tokens so どのように Dollar-Betrag に基づき の aktuellen 模型-Preisliste と gleicht ihn と共に Task-予算 ab.

***Nutzen:*** *Warnt の前に teuren Läufen, まだ bevor の 最初の Cent ausgegeben wird.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**33. Dynamisches Task-予算 と共に 80%-Spar-Downgrade & 100%-Stopp**

**問題:** Ein einzelner komplexer Task konnte unbemerkt gesamte Tagesbudget verzehren.

**Mechanik:** Überwacht kumulierten 費用 pro 任務 (z. B. 限度 $0.25): Schaltet に際して 80 % 自動の auf 一つの günstiges Spar-模型 herunter と pausiert Task に際して 100 % と共に Telegram-Freigabeanfrage.

***Nutzen:*** *Verhindert Kostenausreißer に際して mehrstufigen autonomen Recherchen.*

**価格 & Schattenseite (Trade-off):** Bei Erreichen の 80 % の Task-Budgets となる auf 一つの günstigeres Spar-模型 heruntergeschaltet; に際して 100 % erfolgt の Hard-Stop.

**Wann ausschalten? & Schwellenwerte:** Task-予算 (Std: 0,50 $) のために große アーキテクチャ-Läufe auf 2,00 $ anheben.

**34. Loop-Killer & Max-Steps-Begrenzung (15 段階 / Jaccard > 90%)**

**問題:** Wenn 一つの Agent に際して 一つの fehlerhaften Tool-Aufruf in 一つの Endlosschleife gerät, verbrennt er im Sekundentakt Guthaben.

**Mechanik:** Prüft 各 段階 auf exakte Wiederholungen (>= 3) 又は semantische Kreisbezüge (> 90 % Ähnlichkeit) と kappt 任務 spätestens の後に konfigurierbaren Max-Steps-限度 (標準: 15 段階).

***Nutzen:*** *Schützt 口座 信頼できる の前に unbemerkten Endlosschleifen について Nacht.*

**価格 & Schattenseite (Trade-off):** Zu scharfe Grenzwerte (Max Identisch = 2 又は Jaccard < 0.80) 停止する legitime iterative 任務 どのように schrittweises 試験-Fixing 又は Paginierung zu früh.

**Wann ausschalten? & Schwellenwerte:** Kurzzeitig ausschalten, wenn 一つの Agent bewusst viele gleichförmige Schleifen-段階 (z. B. Massen-移行 の 50 綴) abarbeiten soll. | Schwellenwerte: Max Identisch: 2 = stoppt Retries | > 5 = teure Schleifen (Std: 3). Jaccard: < 0.80 = Fehlalarme | > 0.96 = erkennt Schleifen spät (Std: 0.90). Max 段階: Std 15.

## 4. カーネル、スケジューリングおよびデーモンアーキテクチャ（柱 1）

35. Hermes Master Orchestrator (Hermes-Core)
問題: Unkoordinierte Einzelagenten arbeiten gegeneinander 又は verlieren Gesamtziel aus Augen.
Mechanik: Zentrale 過程-Leitstelle への Auftragsanalyse, Rollenauswahl, 監視 と 成果-受領.
Nutzen: Sichert 戦略的な Kohärenz について 全ての 70 専門家.
価格 & Schattenseite (Trade-off): Mehr aktive Hintergrund-サービス と hohes Reasoning ('high') erhöhen Grundlast と Token-費用.
Wann ausschalten? & Schwellenwerte: Über 14 Sektionen im Tab 'HERMES CORE' granular steuerbar.

**36. Tickless Kernel Daemon & Asyncio Event-Loop (Port 8420 / 8501)**

**問題:** Ständiges Polling im Sekundentakt verschwendet CPU-資源 と Systemenergie.

**Mechanik:** Berechnet exakten Zeitpunkt の nächsten fälligen Tasks ('Sleep-until-next-due') と versetzt Kernel bis dahin in Ruhezustand.

***Nutzen:*** *Minimale Systemlast im Leerlauf に際して verzögerungsfreier Aufwach-Reaktion.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**37. Prioritäts-Preemption Scheduler (P0–P3 Warteschlange)**

**問題:** Dringende Benutzerfragen im Direkt-Chat mussten 保守する, wenn Hintergrund-Jobs 全ての Slots belegten.

**Mechanik:** Priorisierte Warteschlange: P3-任務 (Direkt-Chat / 非常) überholen laufende P1/P2-Routinejobs sofort.

***Nutzen:*** *Garantiert schnelle Antwortzeiten im Dialog にもかかわらず hoher Hintergrundlast.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen

以内に の Moduls).

**38. Frei skalierbare Concurrency-抑制 (backend/concurrency_throttle.py)**

**問題:** Zu viele gleichzeitige エージェント führen zu HTTP-429-Rate-Limits と 99 % CPU-Spitzen.

**Mechanik:** Reguliert aktive Worker について 一つの Asyncio-Semaphore (einstellbar 経由 Slider の 1 bis 16 parallelen Tasks im Guardian と Control Center).

***Nutzen:*** *Erlaubt gedrosselten Schutzbetrieb auf Laptops ebenso どのように Hochlastbetrieb auf Workstations.*

**価格 & Schattenseite (Trade-off):** Knappe Stagnations-Limits (< 30 Sek.) brechen langsame Deep-Reasoning-Modelle (z. B. o3/R1) fälschlich mitten im Denkprozess ab; mehr Worker erhöhen RAM- と API-Spitzenlast.

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して extrem langlaufenden Einzel-Analysen 又は lokalen LLM-Läufen auf langsamer 機材. | Schwellenwerte: Max Worker: 1–2 = Rückstau | > 8 = RAM-Spitzen & HTTP-429 (Std: 4). Stagnation: < 30s = bricht Denker ab | > 180s = späte Heilung (Std: 45s).

**39. Autonomer 30-Sekunden-監視役 & Crash-Recovery (watchdog_daemon.py)**

**問題:** Abgestürzte 又は eingefrorene Tasks blieben dauerhaft im 状態 in_progress hängen.

**Mechanik:** Prüft 全ての 30 Sekunden 全体的 e laufenden 過程, bricht Zombie-Tasks の後に Timeout (120s) ab と überführt verwaiste 任務 in automatische Crash-Recovery.

***Nutzen:*** *Hält 全ての Worker-Slots frei の Karteileichen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**40. Deadlock- & Stagnations-Supervisor (backend/deadlock_supervisor.py)**

**問題:** Gegenseitige 依存 の間に zwei エージェント できる zu 一つの stillen Einfrieren führen.

**Mechanik:** Erkennt zirkuläre Wartezustände と Tasks 無しに 進捗 (> 45s) と löst Blockade autonom と共に exponentiellem Backoff auf.

***Nutzen:*** *Verhindert Pipeline-Stillstände に際して komplexen Gruppenarbeiten.*

**価格 & Schattenseite (Trade-off):** Ein zu knappes Stagnations-限度 (< 30s) bricht langsame Deep-Reasoning-Modelle (o3/R1) mitten im Denkprozess ab.

**Wann ausschalten? & Schwellenwerte:** Bei Nutzung の Reasoning-Modellen Stagnations-限度 auf 90–120s erhöhen (Std: 45s).

**41. Flotten-Selbstheilung & 容器-Auto-Restart (backend/fleet_self_healing.py)**

**問題:** Fällt 一つの の 11 Docker-容器 aus, musste er 手動の 新規の gestartet werden.

**Mechanik:** Überwacht 全ての Core-容器 について abgesicherten docker-socket-proxy と führt に際して Ausfällen (> 60s) autonom 一つの Neustart inklusive 監査-Eintrag durch.

***Nutzen:*** *Sichert unbeaufsichtigten 24/7-運用 の gesamten Docker-Landschaft.*

**価格 & Schattenseite (Trade-off):** Ein dauerhaft defekter 容器 できる を通じて Auto-Restart in 一つの Neustart-Schleife geraten.

**Wann ausschalten? & Schwellenwerte:** Während manueller 容器-Wartungsarbeiten kurzzeitig 無効化する.

**42. Proaktiver Morgen-Standup & Weckdienst (backend/autonomous_standup.py)**

**問題:** Morgens fehlte 一つの kompakte 概要 について nächtliche Läufe, 費用 と anstehende 決定.

**Mechanik:** Generiert 毎日の um 08:00 Uhr 一つの konsolidierten 状態報告 im Obsidian-Vault (01_PROJEKTE/Daily_Briefings/) と pusht ihn an 計器盤 と Telegram.

***Nutzen:*** *Startet 各 Arbeitstag と共に 一つの 製造する 決定稟議.*

**価格 & Schattenseite (Trade-off):** Jedes automatische Morgen-Briefing verbraucht ~1.500–3.000 Tokens への eingestellten Zeitpunkt.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn 無 毎日の Morgen-Report gewünscht は (Std: 08:00 Uhr).

## 5. マルチエージェントプロセスモデル、階層および統合（柱 2）

43. 70 Deklarative エージェント-Profile & Markdown-Personalakten
問題: Generische KI-Assistenten haben 無 Rollenbewusstsein と 無 definierten Qualitätsstandards.
Mechanik: 各 の 70 専門家 は in agent_profiles.json so どのように 一つの eigenen 人事記録 (00_HR_Agenten/) と共に システム-プロンプト, Modellroute, 道具-Scope と SOPs verankert.
Nutzen: Schafft verlässliche virtuelle Fachkräfte のために 戦略, 法務, Bau, 財務, 市場開拓 と IT.
価格 & Schattenseite (Trade-off): Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**44. Robert 委任, Fan-in-Synchronisation & Lisa Marker-Kontrakt (SYNTHESE-BEREIT)**
**問題:** Bei parallelen Teilaufgaben gingen Teilergebnisse verloren 又は となった unvollständig zusammengefasst.
**Mechanik:** Robert zerlegt Großprojekte; 一つの 過程-Lock (_MC_DATA_LOCK) wartet auf 締結 aller Teilaufgaben (Fan-in). Lisa aggregiert 報告 の下に 監査 の Pflicht-Markers SYNTHESE-BEREIT; Robert erteilt in 行 1 formale FREIGABE.
***Nutzen:*** *Mehrstufige 品質確保 の後に Vorbild 一つの echten Unternehmensführung.*
**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**45. Standardisierter Agent-to-Agent Handoff-Desk (backend/handoff_desk.py)**
**問題:** Stille Kontextverluste に際して の 引渡 の 一つの 専門家 an nächsten.
**Mechanik:** Erzeugt formale Übergabeprotokolle と共に Deliverables, offenen Annahmen と SHA-256-Prüfsumme, からの 受取人-エージェント quittiert となる müssen.
***Nutzen:*** *Revisionssichere 接続口 の間に allen 部門.*
**価格 & Schattenseite (Trade-off):** Strikte SLA-Prüfungen weisen sehr kurze, しかし korrekte Faktenantworten zurück, wenn min_content_length zu 高い (> 300) ist.
**Wann ausschalten? & Schwellenwerte:** 標準: 100 文字 Mindestlänge, 60 Min. Turnaround.

**46. Zweistufiger DAG-Scheduler のために parallele 企画-実行 (backend/dag_scheduler.py)**
**問題:** Sequentielle Abarbeitung vieler Teilaufgaben dauert に際して Großprojekten zu lange.
**Mechanik:** Berechnet gerichteten azyklischen Graphen (DAG): Führt unabhängige Recherchen (段階 1) parallel aus と startet danach 自動の 統合 (段階 2).
***Nutzen:*** *Verkürzt dbname を通じた時間 komplexer Analysen um 一つの Vielfaches.*
**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**47. Self-Correction & Tool Execution Reflection Loop**
**問題:** 揮発性の Syntax- 又は Parameterfehler に際して Tool-Aufrufen führten への sofortigen Abbruch.
**Mechanik:** Fängt Tool-障害 ab, reflektiert Fehlermeldung と korrigiert Aufrufparameter autonom in bis zu 3 Versuchen.
***Nutzen:*** *Deutlich höhere Robustheit に際して 綴-, データベース- と Web-Operationen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**48. Unterbrechbare Langzeit-Workflows と共に SQLite State-Snapshotting**

**問題:** Wartet 一つの Task stundenlang auf 一つの 利用者-承認, belegt er unnötig 記憶 又は verliert seinen 状態 に際して 一つの Neustart.

**Mechanik:** Friert exakten Ausführungszustand in の 表 workflow_checkpoints 一つの と nimmt Arbeit の後に 承認 nahtlos 再び auf.

***Nutzen:*** *Ermöglicht mehrtägige Geschäftsprozesse について Neustarts hinweg.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**49. Dual-Agent 合意関門 & Adversarial Critic Review (backend/consensus_gate.py)**

**問題:** Ein einzelner Agent bemerkt eigene Denkfehler 又は Halluzinationen に際して kritischen 任務 oft nicht.

**Mechanik:** Erzwingt に際して P3-Tasks と kritischen Eingriffen 一つの formales Gegenlesen を通じて 一つの unabhängigen 監査者-エージェント (Red-班/Critic). Im Kanban-Board als automatisiertes 四眼 品質関門 verdrahtet (Katharina Völk / Robert Review と共に 1–100% Score と 状態-Lock に際して Revisionsbedarf).

***Nutzen:*** *四-Augen-Prinzip のために 全ての geschäftskritischen 成果.*

**価格 & Schattenseite (Trade-off):** Verdreifacht (3x) Token-Verbrauch と erhöht 応答時刻 um +6 bis +20 Sek.; 全会一致 birgt Deadlock-Gefahr.

**Wann ausschalten? & Schwellenwerte:** Erst ab 優先 P3/P4 有効化する と 2/3-多数 wählen.

## 6. MCP ドライバー、外部サービスおよびマルチモーダルツール（柱 3）

**50. Archon CRM & PocketBase-Ökosystem (Ports 3005, 8090, 8092)**

**問題:** Kundenkontakte, 電子郵便-Historien と 媒体-Assets lagen getrennt の KI-エージェント.

**Mechanik:** Dediziertes CRM-Frontend (Port 3005) と共に PocketBase-Backend (Port 8090) と Media-サーバー (Port 8092), voll angebunden について scripts/archon_crm_mcp.py.

***Nutzen:*** *エージェント できる 連絡先 abrufen, nachrecherchieren, 評価する (Lead-Scoring) と Korrespondenz zuordnen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**51. Automatisches 電子郵便-Matching & LinkedIn-書庫-Importer**

**問題:** Tausende 連絡先 aus LinkedIn-Exporten と 電子郵便-Postfächern mussten 手動の abgeglichen werden.

**Mechanik:** Synchronisiert Postfächer (--fetch-mails), ordnet Mails 自動の bestehenden CRM-Kontakten zu と importiert 完全 e LinkedIn-Connections-CSVs inklusive Deduplizierung.

***Nutzen:*** *Baut in Minuten 一つの完全な, angereicherte Beziehungsdatenbank auf.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**52. Robert Tool-Permission Manager (backend/tool_permission_manager.py)**

**問題:** エージェント すべき に際して fehlenden Python-/Node-Paketen ない hilflos abbrechen, dürfen しかし も

nichts ungefragt 設置する.

**Mechanik:** Robert stellt 一つの formellen Installationsantrag (pending_permission) と共に Befehl と 理由付; erst の後に deiner 1-Klick-承認 となる Paket installiert.

***Nutzen:*** *Kontrollierte Erweiterbarkeit に際して vollem Schutz の Host-Systems.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**53. Event-Driven Webhook-Mesh (backend/event_mesh.py)**

**問題:** 外部の 事象 aus GitHub, Formularen 又は Drittsystemen konnten 無 エージェント-Aktionen auslösen.

**Mechanik:** Universeller Webhook-受取人, の 外部の JSON-Payloads validiert と direkt in ausführbare Kanban-Tasks umwandelt.

***Nutzen:*** *Verbindet Archon OS in Echtzeit と共に の restlichen IT-Landschaft.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**54. Zentraler Multi-Channel Notification Hub (backend/notification_hub.py)**

**問題:** Wichtige 警告 gingen im UI unter, の間に unwichtige 報告 auf Handy störten.

**Mechanik:** Routet 報告 厳密に の後に Schweregrad: INFO -> UI-Toast, WARNING -> 計器盤-Log, CRITICAL -> sofortiger Telegram-Push.

***Nutzen:*** *Zielgenaue Alarmierung 無しに Benachrichtigungs-Müdigkeit.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**55. Multi-Modale Vision- & Screen-Capture-Pipeline (backend/vision_pipeline.py)**

**問題:** Visuelle Probleme 又は 表 in Bildern mussten mühsam beschrieben werden.

**Mechanik:** Direkte Bildschirmaufnahme im Browser (Kamera-Button im Chat) so どのように automatische OCR- と Strukturanalyse のために hochgeladene Bilder (.png, .jpg, .webp).

***Nutzen:*** *エージェント sehen genau das, was du auf Bildschirm siehst.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**56. Apollo Voice Engine — Edge-TTS & Whisper Diktat (apollo_voice_engine.py)**

**問題:** Lange 本文 am Smartphone 又は デスクトップ einzutippen kostet 時間; 報告 möchte man sich unterwegs 聴聞する.

**Mechanik:** Bidirektionale Sprachpipeline: Mikrofon-Diktat (Speech-to-本文) direkt im Chat-Eingabefeld と natürliche Sprachausgabe (本文-to-Speech) のために Briefings. Vollständige Telegram-Voice-統合 (POST /api/telegram/voice-relay と POST /api/voice/synthesize) のために mobile Audio-Memos via edge-tts (de-DE-ConradNeural).

***Nutzen:*** *Verwandelt Archon OS in 一つの sprachgesteuerten Executive Assistant.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**57. Oracle News レーダー & SEO Execution Agent**

**問題:** Marktveränderungen と SEO-Schwachstellen eigener Webseiten となった のみ sporadisch geprüft.

**Mechanik:** Oracle News レーダー scannt kontinuierlich Fachquellen auf Projektrelevanz; の SEO Agent prüft 頁 auf Meta-Tags, Struktur と Keyword-Dichte.

***Nutzen:*** *Kontinuierliche Markt- と Sichtbarkeitsoptimierung im Hintergrund.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

## 7. 記憶、データベース衛生および Obsidian セカンドブレイン（柱 4）

**58. Zentraler SQLite WAL-Memory-Layer (5 Standardtabellen)**

**問題:** Gleichzeitige Lese- と Schreibzugriffe mehrerer エージェント führten に際して einfachen 綴 zu Konflikten.

**Mechanik:** Transaktionale SQLite-データベース (data/agentic_memory.db) im Write-Ahead-Logging-Modus (WAL) と共に 5 Kerntabellen tasks, conversations, learnings, kv_store と user_profile.

***Nutzen:*** *Blockadefreie Parallelzugriffe と共に höchster Datensicherheit.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**59. Prozesssicheres Kanban-Sidecar と共に atomarem Dateiaustausch (backend/kanban_sidecar.py)**

**問題:** Metadaten どのように WIP-Limits 又は Tags konnten に際して parallelen Schreibvorgängen korrumpieren.

**Mechanik:** Schützt config/kanban_meta.json について 一つの prozessweiten Mutex と atomare Write-Replace-Operationen.

***Nutzen:*** *Verhindert Datenverlust に際して hochfrequenten Board-Aktualisierungen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**60. Docker-RAG-Bridge zu AnythingLLM (5.068 indexierte Dokumente auf Port 3001)**

**問題:** im Docker-Vektorspeicher vorhandene 知識 となった に際して normalen Chat-Anfragen oft ない 自動の herangezogen.

**Mechanik:** backend/rag_bridge.py verbindet Agentenkern direkt と共に AnythingLLM-Workspace ARCHON と liefert relevante Textpassagen als Grounded Context.

***Nutzen:*** *Fundierte Antworten auf Basis deiner echten Unternehmensdokumente statt allgemeiner Schätzungen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**61. Dreistufige Projektordner-Disposition im Obsidian-Vault**

**問題:** Wenn エージェント 綴 erzeugen, entsteht 無しに feste 規則 迅速な Unordnung im Vault.

**Mechanik:** Legt のために 各 企画 自動の 3-段階-Struktur an: 01_Zwischenberichte, 02_Medien と 03_Endprodukte.

***Nutzen:*** *Saubere Trennung の間に Rohentwürfen と abgenommenen Endergebnissen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**62. Obsidian Vault Indexer & Automatische Map of 内容 (backend/vault_hygiene.py)**

**問題:** Neue Projektberichte blieben im Obsidian-Graph oft unverlinkt ('Orphan Notes').

**Mechanik:** Scannt Vault, spürt verwaiste Notizen auf と aktualisiert vollautomatisch zentrale _MOC_Projekt_Index.md と共に Wikilinks.

***Nutzen:*** *Hält Wissensgraphen in Obsidian jederzeit 完全な vernetzt.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**63. Autonomes Fact-Harvesting & 85%-Ähnlichkeits-Deduplizierung (backend/post_task_pipeline.py)**

**問題:** Wichtige Erkenntnisse aus abgeschlossenen 任務 gingen の下に 又は となった mehrfach doppelt gespeichert.

**Mechanik:** Extrahiert の後に Task-締結 自動の neue Fakten のために learnings と user_profile と verhindert Dubletten ab 85 % Textähnlichkeit.

***Nutzen:*** *システム となる と共に 各 erledigten 任務 klüger, 無しに 記憶 zuzumüllen.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**64. Automatisches Online-DB-Backup と共に 7-Tage-Rotation (backend/db_backup_manager.py)**

**問題:** Ohne regelmäßige Datenbanksicherungen droht に際して Hardwaredefekten の 損失 aller Tasks と Profile.

**Mechanik:** Führt im laufenden 運用 PRAGMA wal_checkpoint(TRUNCATE) aus と speichert komprimierte .db.gz-Archive in .hermes/backups/db/.

***Nutzen:*** *Lückenlose Datensicherung 無しに 一つの Sekunde Ausfallzeit.*

**価格 & Schattenseite (Trade-off):** Tägliche Voll-Backups belegen je の後に Aufbewahrungsdauer ~50–500 MB Festplattenspeicher.

**Wann ausschalten? & Schwellenwerte:** 標準: 7 Tage 保管 (db_backup_keep_days).

**65. Autonomer SQLite Vacuum & Fragmentierungs-Optimizer (backend/db_optimizer.py)**

**問題:** Gelöschte Datensätze と häufige Updates blähen SQLite-綴 について Monate unnötig auf.

**Mechanik:** Führt について POST /api/system/optimize-db auf allen データベース PRAGMA optimize と VACUUM aus と berechnet freigegebenen Speicherplatz.

***Nutzen:*** *Hält Abfragezeiten dauerhaft im Millisekundenbereich.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**66. Atomares Pre-Write-Snapshotting & 1-Klick-Rollback (backend/snapshot_manager.py)**

**問題:** Wenn 一つの Agent 一つの 綴 fehlerhaft überschreibt, war 復元 の Vorzustands aufwendig.

**Mechanik:** Sichert の前に jedem Schreibzugriff 自動の 原本 in .hermes/snapshots/ と stellt es 経由 Klick 又は API sofort 再び her. 追加 als Pre-更新 Snapshots aller SQLite-データベース の前に GitHub-Pulls と 容器-Updates in data/snapshots/ integriert.

***Nutzen:*** *Vollständiges Sicherheitsnetz に際して allen 綴- と Code-Änderungen.*

**価格 & Schattenseite (Trade-off):** 各 Pre-Write-Snapshot kostet ~5–15 ms Disk-I/O の前に Überschreiben 一つの 綴.

**Wann ausschalten? & Schwellenwerte:** Ringpuffer standardmäßig auf 50 Snapshots begrenzt.

**67. Git-kompatible 実況-Diff-Vorschau (backend/diff_engine.py)**

**問題:** Vor 一つの 承認 möchte man exakt sehen, welche 行 sich ändern, statt ganze 綴 新規の zu lesen.

**Mechanik:** Erzeugt 一つの farbigen Zeilenvergleich (Rot = entfernt, Grün = 新規の) の間に Bestand と エージェン

ト-草案.

***Nutzen:*** *Sekundenschnelle 監査 の Code- と Dokumentenänderungen im HITL-Gate.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

## 8. セキュリティ、ゼロトラストガバナンスおよびシステム診断（柱 6）

**68. Kryptografisch verkettetes SHA-256 監査-Log (backend/audit_log.py)**

**問題:** Einfache 本文-記録 lassen sich nachträglich verändern 又は 削除する.

**Mechanik:** Jedes 事象 となる と共に SHA-256-Hash の Vorgängereintrags verkettet (Blockchain-Prinzip) と はについて GET /api/audit/verify auf Manipulation prüfbar.

***Nutzen:*** *Revisionssicherer 証跡 aller システム- と Freigabeaktionen.*

**価格 & Schattenseite (Trade-off):** Minimaler Schreib-Overhead (~2 ms pro Eintrag); kryptografische Kette wächst について Monate an.

**Wann ausschalten? & Schwellenwerte:** Für revisionssichere Governance dauerhaft eingeschaltet lassen.

**69. Local Security Guard — 25 プロンプト-Injection- & Jailbreak-Signaturen**

**問題:** Bösartige 電子郵便 s 又は Webseiten できる versteckte Befehle enthalten ('Ignore previous instructions...').

**Mechanik:** Scannt 各 externen Texteingang に基づき の 25 Angriffsmustern, bevor er in Kontext 一つの エージェント gelangt.

***Nutzen:*** *Wehrt indirekte プロンプト-Injections an の Systemgrenze ab.*

**価格 & Schattenseite (Trade-off):** Sehr strenge Heuristiken できる に際して IT-Security-Texten 又は Pen-Testing-Prompts Fehlalarme (False Positives) auslösen.

**Wann ausschalten? & Schwellenwerte:** Nur のために autorisierte Red-Teaming-試験 kurzzeitig lockern.

**70. Secret-Masking & Red-Teaming Output Guard (backend/secret_masker.py)**

**問題:** API-Schlüssel 又は Passwörter könnten versehentlich in 記録, Berichten 又は Chat-Ausgaben auftauchen.

**Mechanik:** Filtert ausgehende 本文 in Echtzeit auf Schlüssel-Muster (sk-..., Bearer, Passwörter) と ersetzt sie を通じて [GESCHÜTZTER_KEY].

***Nutzen:*** *Verhindert versehentliche Offenlegen vertraulicher Zugangsdaten.*

**価格 & Schattenseite (Trade-off):** Aggressive Regex-Maskierung できる in seltenen Fällen harmlose Base64- 又は Hash-Strings in Code-Ausgaben schwärzen.

**Wann ausschalten? & Schwellenwerte:** Im Produktivbetrieb への Schutz の API-鍵 dauerhaft 有効な lassen.

**71. Abgesicherter Docker-Socket-Proxy (Port 2375 intern)**

**問題:** Direkter 利用 auf /var/run/docker.sock gibt Containern uneingeschränkte Root-権利 について Host-システム.

**Mechanik:** Schaltet 一つの restriktiven Proxy-容器 dazwischen, の のみ explizit erlaubte 状態- と Neustart-Befehle durchlässt.

***Nutzen:*** *Schützt Host-Betriebssystem の後に Zero-Trust-Prinzip.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**72. A/B-模型-Benchmarker & Empirischer Performance-Scorer (backend/model_benchmarker.py)**

**問題:** Neue Sprachmodelle erscheinen wöchentlich; ihre Eignung のために eigene Unternehmensaufgaben bleibt

oft unklar.

**Mechanik:** Testet zwei Modelle parallel と共に identischen 任務 と vergleicht 応答時刻, Token-費用 と Ergebnisqualität.

***Nutzen:*** *Objektive Datenbasis のために Modellwahl im エージェント部隊.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**73. Systemweiter 360°-Health-Score & Diagnose-Engine (backend/system_diagnostics.py)**

**問題:** Einzelne Metriken zeigen oft ない auf 一つの Blick, どのように gesund Gesamtsystem 真に ist.

**Mechanik:** Berechnet aus Docker-Flottenstatus (35%), Concurrency-Queue (20%), 監査-完全性 (20%), Deadlock-Freiheit (15%) と RAG-Erreichbarkeit (10%) 一つの Gesamtscore の 0–100 % 含む Handlungsempfehlungen.

***Nutzen:*** *Sofortige Gesamtdiagnose の Betriebssystems について GET /api/system/health-score.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

## 9. 自己進化、マーケットプレイスおよび自動文書化（柱 7）

**74. 自律的な Skill-統合 と共に Personalakten-Rückkopplung (backend/skill_synthesizer.py)**

**問題:** Erfolgreiche mehrstufige Lösungswege mussten bisher 手動の als neuer Skill programmiert werden.

**Mechanik:** Erkennt wiederkehrende erfolgreiche Tool-Ketten, generiert 自動の 一つの SKILL.md-草案, trägt ihn in 人事記録 の エージェント 一つの と legt ihn への 承認 vor.

***Nutzen:*** *システム 高度な seine 能力 im 運用 selbstständig.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**75. Dynamischer Talent- & Skill-市場 (skill_markt.js & AgentSkillsHub)**

**問題:** Neue 役割 又は Workflows のために neue Geschäftsfelder の Grund auf 新規の zu schreiben, は zeitintensiv.

**Mechanik:** Durchsuchbarer Katalog への 1-Klick-搬入 と -搬出完全 er エージェント-Profile so どのように externer Skills (含む agentskillshub.dev 統合).

***Nutzen:*** *Skaliert エージェント-Flotte に際して neuen Projekten in Sekunden.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**76. Vollständige Frontend-Backend-Synchronisation (238 Fetches / 581 Pfade / 29 Modale)**

**問題:** In großen Web-応用 führen verwaiste Buttons 又は unverbundene Modale oft zu stillen Fehlern.

**Mechanik:** Automatisierte Prüf-Pipeline stellt sicher, dass 全ての 238 Frontend-API-Aufrufe と 29 Modale exakt と 共に 368 Backend-Routen verdrahtet sind.

***Nutzen:*** *Garantiert 0 tote Links と 100 % bedienbare Steuerelemente in allen 観覧.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

**77. Autonomer Master-Dossier- & 便覧-Generator (backend/system_dossier_generator.py)**

**問題:** アーキテクチャ-Dokumentationen veralten meist 既に wenige Tage の後に ihrer Erstellung.

**Mechanik:** Generiert auf Knopfdruck (POST /api/system/generate-dossier) aktuelle システム-アーキテクチャ-Dossier simultan im Projektordner と im Obsidian-Vault (00_DOKUMENTATION/).

***Nutzen:*** *Hält Systemdokumentation 無しに manuellen Pflegeaufwand dauerhaft auf neuesten Stand.*

**価格 & Schattenseite (Trade-off):** Als permanente Kern-観覧 bzw. システム-Infrastruktur im 計器盤 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-費用 entstehen のみ に際して aktiver 実行 の LLM-Aktionen 以内に の Moduls).

## 10. 参照マトリクス：制御可能な Evey/Hermes プラグイン

| Nr. & Plugin-ID | Fach-分類 | Kernmechanik & 機能 im Betriebssystem |
|---|---|---|
| **78. evey-cost-guard** | セキュリティ & Wächter | Echtzeit-Kostenüberwachung と harte 予算-Abschaltung pro Tag と Task. |
| **79. evey-session-guard** | セキュリティ & Wächter | Überwacht Sitzungslaufzeiten, Token-Obergrenzen と verhindert Zombie-会議. |
| **80. evey-email-guard** | セキュリティ & Wächter | Scannt eingehende 電子郵便 s の前に の Triage auf Phishing と プロンプト-Injections. |
| **81. evey-watchdog** | セキュリティ & Wächter | Prüft im 30s-Takt aktive 過程 と startet blockierte Worker 自動の 新規の. |
| **82. evey-cache** | Performance & Cache | 24h-応答-Cache への sofortigen, kostenfreien Beantwortung wiederholter Abfragen. |
| **83. disk-cleanup** | Performance & 保守 | Bereinigt temporäre Artefakte, veraltete 記録 と gibt Festplattenspeicher frei. |
| **84. evey-telemetry** | Monitoring & 分析 | Erfasst Token-Ströme, Latenzen と 模型-Aufrufe のために Mission Control と Guardian. |
| **85. evey-status** | Monitoring & 分析 | Kontinuierlicher Health-Ping auf 全ての 11 Docker-容器 と Gateways. |
| **86. evey-delegation-score** | Monitoring & 分析 | Bewertet Erfolgs- と Qualitätsquote の Aufgabenübergaben エージェント間で. |
| **87. evey-autonomy** | エージェント-Intelligenz | Steuert Autonomiegrad に際して mehrstufigen Hintergrund-決定. |
| **88. evey-council** | エージェント-Intelligenz | Mehrheits- と Konsensbildung の間に mehreren 専門家 (Mastermind-Kopplung). |
| **89. evey-reflect** | エージェント-Intelligenz | Selbstreflexions-Schleife への eigenständigen Fehlerkorrektur の前に Ergebnisabgabe. |
| **90. evey-learner** | エージェント-Intelligenz | Extrahiert dauerhafte Erkenntnisse aus erledigten 任務 のために learnings-表. |

| | | |
|---|---|---|
| **91. evey-identity** | エージェント-Intelligenz | Sichert strikte Einhaltung の SOUL.md, Tonalität と 役割-Leitplanken. |
| **92. evey-habits** | エージェント-Intelligenz | Verwaltet wiederkehrende Routinen どのように 毎日の 08:00-Uhr-Standup-Briefing. |
| **93. evey-proactive** | エージェント-Intelligenz | Erlaubt エージェント, に際して erkannten Engpässen selbstständig Handlungsvorschläge zu machen. |
| **94. evey-goals** | Planung & Tasks | Verknüpft operative Einzelaufgaben im Kanban と共に übergeordneten Unternehmenszielen. |
| **95. evey-scheduler** | Planung & Tasks | 時間- と prioritätsgesteuerte Einplanung の Hintergrund-Aufträgen. |
| **96. evey-bridge** | Brücken & 接続口 | 内部 Datendrehscheibe の間に FastAPI-Monolith, Hermes-Daemon と Sidecars. |
| **97. evey-mqtt** | Brücken & 接続口 | Leichtgewichtiges Nachrichtenprotokoll のために IoT- と Echtzeit-Statussignale. |
| **98. evey-moltbook** | Brücken & 接続口 | Synchronisiert strukturierte Arbeitsbücher と 統合-Notizen. |
| **99. evey-telegram-ux** | Brücken & 接続口 | Steuert 呼称-Routing (/robert, /anwalt) と 対話型の 承認-Buttons in Telegram. |
| **100. evey-github** | Brücken & 接続口 | Anbindung のために Repository-Operationen, Commit-Analysen と Issue-Tracking. |
| **101. evey-research** | Tools & Recherche | Tiefensuche-ドライバー のために Web-Scraping と 情報源-Extraktion im Research Lab. |
| **102. evey-news** | Tools & Recherche | Hintergrund-Scanner のために Branchen-Feeds と Markt-Neuigkeiten (Oracle News レーダー). |
| **103. evey-digest** | Tools & Recherche | Komprimiert umfangreiche Dokumentensammlungen in kompakte Management-Summaries. |
| **104. evey-sandbox** | Tools & Execution | Isolierte Ausführungsumgebung のために generierten Python- と 分析-Code im Showroom. |
| **105. evey-rag** | 記憶 & 知識 | Vektor-Retrieval-ドライバー への Anbindung の AnythingLLM と lokalen Wissensarchiven. |
| **106. evey-memory-adaptive** | 記憶 & 知識 | Dynamische 加重 の Kurzzeit- と Langzeitgedächtnis je の 後に Kontextdruck. |
| **107. evey-memory-consolidate** | 記憶 & 知識 | Nachtlauf への Zusammenführung と Deduplizierung verwandter Gedächtniseinträge. |
| **108. evey-wallet** | 財務 & Budgets | Verwaltet API-Guthaben, Kontingente と kostenbezogene Transaktions-Limits. |
| **109. evey-validate** | 品質 & 実行 | Prüft Ausgaben auf Schema-適合, Pflicht-Marker (SYNTHESE-BEREIT) と Formatvorgaben. |

| | | |
|---|---|---|
| **110. evey-verification** | 品質 & 実行 | Faktencheck と Gegenprüfung kritischer Aussagen の前に の Endfreigabe. |
| **111. evey-delegate-model** | 品質 & 実行 | Wählt に際して Teilaufgaben 自動の kosten- と leistungsoptimale Zielmodell. |

## 11. エンタープライズ・レジリエンスアーキテクチャ

**78. 再演 & 時間旅行検誤 (backend/time_travel_debugger.py)**

**問題:** Bei komplexen エージェント-Workflows と共に vielen Tool-Calls は nachträgliche Fehlerdiagnose im Textlog unübersichtlich と Verzweigen の Ausführungssträngen unmöglich.

**Mechanik:** Speichert 各 Teilschritt と共に Ein-/Ausgaben, Snapshots と Tokenverbrauch in 'execution_traces'. Ermöglicht interaktives 再演 (GET /api/traces/{task_id}), State-Forking ab 段階 N (POST /api/traces/{task_id}/fork) と 1-Klick-Rollback modifizierter 綴.

**Nutzen:** Schnelle 原因分析 と risikofreie Was-wäre-wenn-Experimente と共に alternativen Prompts 又は Modellen 無しに Neustart の gesamten Pipeline.

**価格 & Schattenseite (Trade-off):** Erhöht Disk-I/O um ~5–15 ms pro 綴-Edit と belegt je の後に Snapshot-限度 ~50–250 MB zusätzlichen Speicherplatz auf の Festplatte.

**Wann ausschalten? & Schwellenwerte:** Ausschalten auf Systemen と共に extrem knappem Disk-記憶 又は に際して massenhaften 綴-Generierungen (z. B. Build-Artefakte). | Schwellenwerte: Max Snapshots: < 10 = kurze Historie | > 200 = höherer Disk-Verbrauch (Std: 50). Max Trace-Steps: Std 100.

**79. 複数模型 評価行列 & A/B-Benchmarker (backend/model_benchmarker.py)**

**問題:** 模型-Zuweisungen an エージェント basieren mangels empirischer Vergleichswerte oft auf subjektiven Annahmen.

**Mechanik:** Sendet denselben プロンプト simultan an zwei konfigurierte LLMs (POST /api/router/benchmark), misst 遅延, Token-Verbrauch と Antwortlänge と speichert Historie persistent in 'model_benchmarks' (GET /api/router/benchmark/history).

**Nutzen:** Objektive, datengestützte Modellauswahl je Agentenrolle (z. B. DeepSeek V3 のために Schnelligkeit/費用 vs. Claude 3.7 のために komplexe Architekturen).

**価格 & Schattenseite (Trade-off):** N Kandidaten-Modelle kosten N-fache API-Tokens pro Testlauf.

**Wann ausschalten? & Schwellenwerte:** Wann ausschalten? Nur gezielt のために Modellvergleiche 開始する, ない im Dauerbetrieb.

**80. Tri-Agent 合意関門 & 仲裁 (backend/consensus_gate.py)**

**問題:** Kritische Eingriffe (Code-Refactorings, Migrationen) bergen に際して Einzelentscheidungen 危険 の Halluzinationen 又は Sicherheitslücken.

**Mechanik:** Beruft 一つの dreiköpfiges Gremium (システム Architect, Security Officer, QA Lead) 一つの (POST /api/consensus/council-vote). 各 監査者 votiert unabhängig; 承認 erfordert 2/3-多数. 全て 決議 となる in 'consensus_decisions' archiviert (GET /api/consensus/history).

**Nutzen:** Verlässliche Risikominimierung と Eliminierung の Single-Point-of-Failure-Halluzinationen に際して sensiblen Systemänderungen.

**価格 & Schattenseite (Trade-off):** Verdreifacht (3x) Token-Verbrauch と erhöht 応答時刻 um +6 bis +20 Sek. Bei Einstimmigkeits-Zwang droht Entscheidungs-停止 (Deadlock).

**Wann ausschalten? & Schwellenwerte:** Ausschalten (oder erst ab 優先 4 = 重大 有効化する), wenn Geschwindigkeit と niedrige API-費用 Vorrang haben. | Schwellenwerte: Ab 優先: Prio 2 = fast 各 Task kostet 3x Tokens | Prio 4 = のみ kritische Tasks (Std: Prio 3). 多数: 2/3-多数 empfohlen.

**81. 自律的な 任務自己修復 と共に Git-Rollback (backend/task_self_healer.py)**

**問題:** Syntaxfehler 又は fehlerhafte Code-Imports blockieren Tasks sofort と erfordern manuelles Eingreifen の Nutzers.

**Mechanik:** Isoliert Stacktrace, führt に際して Laufzeitfehlern 一つの Git-Rollback modifizierter 綴 を通じて と übergibt エージェント 一つの strukturierten 修正-プロンプト (POST /api/healing/attempt). Bis zu 3 autonome Reparaturversuche; Log-Abruf via GET /api/healing/logs.

**Nutzen:** Maximale 強靭性 に際して Code- と Skript-Ausführungen と共に automatischer Selbstreparatur の前に の 昇格 ins HITL-Board.

**価格 & Schattenseite (Trade-off):** 各 Selbstheilungsversuch kostet 1 zusätzlichen LLM-Call (+3 bis +12 Sek. 遅延 & Token-費用). Ein automatisches Git-Rollback verwirft に際して Fehlschlag も nützliche Teilerfolge.

**Wann ausschalten? & Schwellenwerte:** Git-Rollback ausschalten, wenn unfertige Code-Entwürfe への manuellen Weiterarbeit im Arbeitsverzeichnis stehen bleiben sollen. | Schwellenwerte: Max Reparaturversuche: 1 = gibt sofort auf | > 5 = teure Reparatur-Kaskade (Std: 3).

### 82. 事象網 & Posteingangs-目録-Watcher (backend/event_mesh.py)

**問題:** 任務 mussten bisher 手動の について 計器盤 angestoßen 又は について zeitgesteuertes IMAP-Polling abgefragt werden.

**Mechanik:** Zentraler 内向きの-Event-Bus のために 外部の Webhooks (POST /api/events/inbound/{channel}) と automatischer 目録-Watcher (POST /api/events/scan-directory), の neue 綴 in 'data/inbox/' direkt in Kanban-Triage-Tasks überführt.

**Nutzen:** Vollautomatische Event- と Belegerfassung aus externen Systemen (GitHub, n8n, Stripe, 地域の Dateiscanner) 無しに Benutzerinteraktion.

**価格 & Schattenseite (Trade-off):** Sehr kurze Scan-Intervalle (< 3 Sek.) erzeugen kontinuierliche Dateisystem-Polls; 毎日の Morgen-Standup verbraucht ~1.500–3.000 Tokens pro Lauf.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn の 地域の Inbox-挟 ない 有効な genutzt となる 又は 無 morgendlicher Briefing-報告 gewünscht ist. | Schwellenwerte: Scan-Intervall: 1–2s = unnötige Disk-Polls | > 30s = verzögerte 綴-Erkennung (Std: 5s). Standup-時間: Std 08:00 Uhr.

## 12. 自律性・サンドボックス・ガバナンスアーキテクチャ

### 83. 揮発性の 符号砂場 (backend/ephemeral_sandbox.py)

**問題:** 実行 の エージェント-Code birgt 危険 の Systemabstürzen, Endlosschleifen 又は unbeabsichtigten Dateiänderungen auf Host.

**Mechanik:** Führt Code in kurzlebigen, isolierten Docker-Micro-Containern と共に harter 資源-Begrenzung (512 MB RAM, 1 CPU, 30s Timeout, 無 Netz) aus (POST /api/sandbox/execute). Sandbox となる の後に 締結 rückstandslos gelöscht.

**Nutzen:** Absolute セキュリティ と Systemstabilität に際して の automatisierten Code-実行 と -Erprobung.

**価格 & Schattenseite (Trade-off):** 容器-Kaltstart kostet ~300–800 ms Zusatzlatenz pro Tool-Call. は 通信網 deaktiviert, schlagen pip install 又は 外部の API-Abrufe im Skript sofort fehl.

**Wann ausschalten? & Schwellenwerte:** Ausschalten (oder 通信網 有効化する), wenn vertrauenswürdige 地域の Skripte direkten Host-利用 又は schnelle Paketinstallationen benötigen. | Schwellenwerte: RAM-限度: < 256 MB = OOM-Kill に際して Pandas/NumPy | > 2048 MB = Host-RAM-危険 (Std: 512 MB). Timeout: < 10s = bricht Builds ab (Std: 30s).

### 84. 夜間の 記憶-統合 (backend/memory_consolidator.py)

**問題:** Stetig anwachsende Traces, Heartbeats と veraltete Dialogdaten verlangsamen Datenbankabfragen im Dauerbetrieb.

**Mechanik:** Prunt in Ruhephasen (標準 03:00 Uhr) veraltete Traces と unkritische 記録 に基づき konfigurierbarer Retention-期限 (settings.json). 完全 steuerbar について Hermes Core 設定 (時刻, Retention-Tage, Pruning-Tiefe,

manueller Run-Button).

**Nutzen:** Dauerhaft schlanke データベース, reduzierte Latenzen と automatische システム-衛生 無しに manuellen Wartungsaufwand.

**価格 & Schattenseite (Trade-off):** Während の Konsolidierungslaufs (~10–30 Sek.) entsteht kurze CPU-/Disk-Last; 一つの zu niedrige Pruning-Schwelle (< 0.88) できる verwandte, しかし eigenständige Fakten 要約する.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn lückenlose Roh-Traces のために 一つの forensisches 監査 dauerhaft ungefiltert erhalten bleiben müssen. | Schwellenwerte: 保管: < 7 Tage = 損失 wichtiger 監査-Spuren | > 60 Tage = große SQLite-DB (Std: 14 Tage). Pruning-Schwelle: 0.95 = konservativ | 0.88 = aggressiv (Std: 0.92).

### 85. 視覚の Web 要員 と共に 画面監査 (backend/visual_web_agent.py)

**問題:** 任務 auf komplexen Webportalen 無しに API (SPAs, 書式, 対話型の Dashboards) できる と共に 本文-Scrapern ない bedient werden.

**Mechanik:** Steuert 一つの echten Headless-Chromium-Browser (POST /api/browser/browse-and-capture), erfasst Screenshots のために 各 Klick- と Eingabeschritt in .hermes/visual_traces/{task_id}/ と extrahiert strukturierte Daten.

**Nutzen:** Lückenlose visuelle 追跡可能性 と automatisierte Interaktion と共に modernen Webanwendungen 無しに API-Zugang.

**価格 & Schattenseite (Trade-off):** Headless-Chromium benötigt pro aktivem 監査-Lauf ~300 MB RAM と ~3–5 Sek. Renderzeit.

**Wann ausschalten? & Schwellenwerte:** Wann ausschalten? Bei reinem Backend-運用 無しに UI-Prüfungen.

### 86. 仮想の War-Room & 事前葬儀 圧力試験 (backend/war_room_simulator.py)

**問題:** Unvorhergesehene 安全, Akzeptanz- 又は 遵守-危険 となる に際して Produkteinführungen oft erst の後に Rollout bemerkt.

**Mechanik:** Simuliert drei gegnerische Parteien (Kritischer Großkunde, Offensiver Pentester, Strenger 遵守-Auditor) の前に の 承認 (POST /api/simulation/war-room). Erstellt 一つの aggregierte 危険-Matrix (LOW bis CRITICAL) と共に verbindlichen 付帯条件.

**Nutzen:** Proaktives Aufdecken の Schwachstellen と Vermeidung kostspieliger Fehlentscheidungen の前に Livegang.

**価格 & Schattenseite (Trade-off):** Kostet ~1.500–3.000 zusätzliche Tokens と +4 bis +10 Sek. 待機時刻 の前に eigentlichen Task. Bei Block-Schwelle MEDIUM entstehen häufiger Fehlalarme.

**Wann ausschalten? & Schwellenwerte:** Auto-事前葬儀 im normalen Entwicklungsalltag ausschalten と のみ のために produktionskritische Eingriffe (HIGH/CRITICAL) nutzen. | Schwellenwerte: Block-Schwelle: MEDIUM = blockiert oft vorsorglich | CRITICAL = stoppt のみ katastrophale 危険 (Std: HIGH).

### 87. SLA- & 引渡契約 エージェント間で (backend/sla_contract_enforcer.py)

**問題:** Lose Delegationen in Multi-エージェント-Hierarchien führen oft zu unvollständigen Zuarbeiten, an menschlichen Operator eskaliert werden.

**Mechanik:** Schließt に際して internen Delegationen digitale 契約 と共に Pflichtfeldern, Mindestlänge と Akzeptanzkriterien ab (POST /api/contracts/create). Validiert Lieferungen autonom (POST /api/contracts/validate) と weist mangelhafte 成果 自動の への 追完 ab.

**Nutzen:** Strikte Einhaltung の Qualitätsstandards im エージェント-班 と Vermeidung unfertiger Zwischenstufen im HITL-Board.

**価格 & Schattenseite (Trade-off):** Wird Mindestlänge zu 高い angesetzt (> 300 文字), となる 精密な Kurzantworten 却下された と lösen teure Nacharbeits-Schleifen aus.

**Wann ausschalten? & Schwellenwerte:** Auto-Reject ausschalten, wenn Subagenten も knappe 状態- 又は Ja/Nein-Antworten zurückgeben dürfen. | Schwellenwerte: Min. Länge: < 30 = lässt Platzhalter を通じて | > 300 = lehnt knappe Faktenantworten ab (Std: 100 文字). Max Turnaround: Std 60 Min.

### 88. Zentrale 自律- & 敷居値-制御 (backend/autonomy_settings.py)

**問題:** Feste Schwellenwerte im Programmcode 防ぐ 一つの schnelle 適合 の 安全, Sandbox- と 強靭性-規則 im

laufenden 運用.

**Mechanik:** Bündelt 全ての 10 自律-群 (Concurrency, Loop-Killer, Selbstheilung, Docker-Sandbox, Time-Travel-Snapshots, 合意関門, War-Room, SLA-契約, Inbox/Standup と Flotten-衛生) in settings.json と synchronisiert sie について GET/POST /api/settings/autonomy-modules と共に System Control Center.

**Nutzen:** Jedes Schutz- と 自律-モジュール lässt sich への 存続期間 経由 Klick ein- 又は ausschalten と exakt auf gewünschte 危険- と Ressourcenprofil feinjustieren.

**価格 & Schattenseite (Trade-off):** Ein defekter 容器 できる を通じて Auto-Restart in 一つの Crash-Loop geraten; の visuelle Browser-監査 startet Chromium (+300 MB RAM, +4 Sek. 所要).

**Wann ausschalten? & Schwellenwerte:** Auto-Restart の間に manueller 容器-保守 ausschalten; Browser-監査 に際して reinem Backend-運用 無効化する. | Schwellenwerte: DB-Backup-Tage: < 3 Tage = knappe Historie | > 30 Tage = ~500 MB+ Backup-記憶 (Std: 7 Tage).

## 13. パフォーマンスコア、スループットスケーリングおよびトークン効率

**89. Semantisches プロンプト- & Response-Caching (backend/semantic_cache.py)**

**問題:** Wiederkehrende 又は leicht umformulierte エージェント-Anfragen verursachen unnötige LLM-API-費用 と sekundenlange Wartezeiten.

**Mechanik:** Berechnet のために 各 プロンプト 一つの SHA-256-Hash so どのように 一つの L2-normalisierten 語-/3-Gramm-Vektor in SQLite と liefert に際して 一つの Cosine-Ähnlichkeit >= 0.95 sofort gespeicherten Response-Treffer.

**Nutzen:** Antwortzeiten の下に 10 Millisekunden と 100 % Token-Ersparnis に際して semantisch identischen Anfragen.

**価格 & Schattenseite (Trade-off):** は Cosine-Schwelle zu 低い (< 0.92), übersieht の Cache feine Bedeutungsunterschiede (z. B. 'lösche 試験-DB' vs. 'lösche Prod-DB') と liefert falsche 又は veraltete Antworten.

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して 任務 と共に Echtzeit-Daten (Börsenkurse, 実況-記録, Systemstatus) 又は wenn 各 応答 frisch generiert となる muss. | Schwellenwerte: Cosine-Schwelle: < 0.92 = falsche Antworten aus Cache! | > 0.98 = kaum Treffer (Std: 0.95). TTL: Std 24h. Max Einträge: Std 500.

**90. Paralleles Map-Reduce Sub-Tasking / Fan-Out & Fan-In (backend/map_reduce_executor.py)**

**問題:** Sequenzielle Abarbeitung vieler Einzeldokumente 又は Teilfragen führt zu linearem Zeitwachstum と langen Wartezeiten.

**Mechanik:** Teilt 一覧-注文 自動の auf bis zu N parallele Asyncio-Worker (Semaphore-gesteuert) auf と führt Teilergebnisse im Reduce-段階 zusammen.

**Nutzen:** Vervielfacht 吐出量 に際して Dokumenten-, Recherche- と Code-Batch-Analysen 無しに Überlastung の Event-Loop.

**価格 & Schattenseite (Trade-off):** Erzeugt N-fache gleichzeitige API-Last (Gefahr の HTTP-429 Rate-Limits に際して 提供者) と ~20–35 % Token-Overhead を通じて anschließende Reduce-統合.

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して strikten API-Rate-Limits 又は wenn Teilaufgaben eng voneinander abhängen と sequenziell aufeinander aufbauen müssen. | Schwellenwerte: Max Worker: > 8 = Rate-限度-Gefahr (Std: 4). Chunk-Timeout: Std 30s. Ab Elementen: Std 3.

**91. Speculative Fast-Routing & 遅延-Hedging (backend/speculative_hedger.py)**

**問題:** Einzelne langsame Cloud-Requests (Tail-Latency-Spikes) blockieren gesamte エージェント-実行.

**Mechanik:** Startet の後に 流れ の hedge_delay_seconds (z. B. 2.5s) 自動の 一つの parallelen Ersatz-Call auf 一つの schnellen Fallback-模型 と übernimmt まず eintreffende 成果.

**Nutzen:** Eliminiert Hänger を通じて langsame API-Endpunkte と stabilisiert Antwortzeiten im 99. Perzentil.

**価格 & Schattenseite (Trade-off):** Wenn Primärmodell kurz の後に Hedge-Start doch まだ antwortet, となる beide Requests abgerechnet (bis zu 2x Token-費用 に際して langsamen Prompts).

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して knappem API-予算 又は wenn langsame Reasoning-Modelle genutzt werden, regulär > 10 Sek. Denkzeit benötigen. | Schwellenwerte: Hedge-Verzögerung: < 1.5s = fast 常に doppelte API-費用! | > 8s = kaum 遅延-Schutz (Std: 2.5s).

**92. Dynamisches Context-Compaction & Hierarchisches Summarizing (backend/context_compactor.py)**

**問題:** Lange エージェント-会議 blähen Kontextfenster auf, verlangsamen プロンプト-Prefill と erhöhen 費用 pro Turn drastisch.

**Mechanik:** Extrahiert ab 一つの einstellbaren Token-敷居値 全ての harten Fakten (目標, Dateipfade, 決定, 障害) in 一つの kompakten [HIERARCHICAL_MEMORY_STATE]-Block と behält のみ jüngsten Runden im Volltext.

**Nutzen:** Spart 65–80 % Kontext-Tokens に際して langen 会議 無しに 損失 wichtiger Projektdaten.

**価格 & Schattenseite (Trade-off):** Komprimierungslauf kostet einmalig ~1.000 Tokens; exakte Code-行 又は kleine Detailwerte aus weit zurückliegenden 通知 できる in の 要約 verloren gehen.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn の Agent wortwörtlich auf sehr frühe 通知 derselben 会議 zurückgreifen muss. | Schwellenwerte: Token-Schwelle: < 3000 = vergisst 詳細 zu früh | > 24000 = hohe プロンプト-費用 (Std: 6000). 最終 Turns behalten: Std 4.

**93. SQLite WAL Connection-Pool & Async Write-Batching (backend/async_write_batcher.py)**

**問題:** Hunderte einzelne SQLite-INSERTs に際して parallelen エージェント-Läufen erzeugen Disk-I/O-Spitzen と 'database is locked'-Konflikte.

**Mechanik:** Sammelt 遠隔測定- と 監査-事象 in 一つの Thread-sicheren RAM-Ringpuffer と schreibt sie gebündelt via executemany() im WAL-Modus.

**Nutzen:** Reduziert Festplatten-Schreibzugriffe um rund 80 % と hält 主要-Event-Loop frei の Blockaden.

**価格 & Schattenseite (Trade-off):** Im Falle 一つの harten Stromausfalls 又は kill -9 の前に nächsten Flush-Intervall gehen im RAM gepufferten Einträge の letzten 1–5 Sekunden verloren.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn 各 einzelne Log-Eintrag sofort と 無しに Sekundenverzögerung synchron auf の Festplatte liegen muss. | Schwellenwerte: Batch-Größe: < 5 = viele Disk-Writes | > 100 = mehr 損失 に際して Crash (Std: 25). Flush-Intervall: Std 2.0s.

## 14. 最新式メモリ・RAG アーキテクチャ

**94. Hybrid-Search と共に BM25 FTS5 & Reciprocal Rank 合併 (backend/hybrid_search_engine.py)**

**問題:** Reine Vektorsuche findet semantisch ähnliche 本文, scheitert しかし oft an exakten Fehlercodes, Funktionsnamen 又は Seriennummern.

**Mechanik:** Kombiniert SQLite FTS5 Volltext-Ranking (BM25) と Cosine-Vektor-Ähnlichkeit について Reciprocal Rank 合併 (RRF).

**Nutzen:** Vereint exakte Keyword-Treffer と semantisches Verständnis in 一つの einzigen, hochpräzisen Trefferliste.

**価格 & Schattenseite (Trade-off):** Doppelte 索引-照会 erhöht Such-遅延 um ~15–40 ms; 一つの zu hohes Top-K (> 10) füllt エージェント-プロンプト と 共に tausenden Zusatz-Tokens.

**Wann ausschalten? & Schwellenwerte:** Fast 決してない ausschalten — höchstens Top-K auf 3–5 reduzieren, wenn Token-費用 pro 要求 minimiert となる sollen. | Schwellenwerte: RRF-定数 K: Std 60. BM25-Gewicht: 0.0 = のみ Vektor | 0.5 = Balance | 1.0 = のみ Stichwort (Std: 0.50). Top-K: Std 5.

**95. Lokaler Knowledge-Graph / Tripel-記憶 (backend/knowledge_graph_store.py)**

**問題:** Flache 本文-Chunks erfassen 無 strukturierten 依存 (wer verwaltet welches システム 又は welche Komponente nutzt welchen サービス).

**Mechanik:** Extrahiert と speichert Subjekt-Prädikat-Objekt-Tripel と共に Konfidenzscore in SQLite と ermöglicht mehrstufige Graph-Traversierungen (Multi-Hop).

**Nutzen:** エージェント 検出する システム- と Organisationszusammenhänge について mehrere Ebenen hinweg.

**価格 & Schattenseite (Trade-off):** Automatische Tripel-Extraktion kostet に際して 保存 ~500 Zusatz-Tokens. Ab > 2 Traversal-Hops explodiert Anzahl の Nachbarknoten exponentiell.

**Wann ausschalten? & Schwellenwerte:** Auto-Extraktion ausschalten に際して reinem Kurzzeit-Chat 無しに langfristigen 知識構築; Hops auf 最大 2 begrenzen. | Schwellenwerte: Max Hops: 1 = direkte Nachbarn | 2 = optimal | >= 3 = Knoten-Explosion! (Std: 2). Min. Konfidenz: Std 0.70.

### 96. Agentic Corrective RAG / CRAG (backend/corrective_rag.py)

**問題:** Klassisches RAG reicht irrelevante Suchtreffer ungeprüft an Sprachmodell weiter と provoziert dadurch Fehlantworten.

**Mechanik:** Bewertet 各 abgerufenen Dokument-Chunk に対して 一つの 関連性-敷居値 (0.45) と führt に際して schwachen Treffern 自動の 一つの Query-Rewrite と共に Synonym-拡張 durch.

**Nutzen:** Filtert Wissensrauschen の前に の Antwortgenerierung heraus と erhöht Trefferqualität に際して unklaren Fragen.

**価格 & Schattenseite (Trade-off):** 各 Query-Umformulierung kostet 1 zusätzlichen LLM-Call (+1,5 bis +4 Sek. 遅延). は 関連性-Schwelle zu 高い (> 0.70), dreht CRAG unnötige Extrarunden.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn 知識庫 klein と sauber gepflegt は 又は schnelle Antwortzeiten の下に 2 Sekunden gefordert sind. | Schwellenwerte: 関連性-Schwelle: < 0.30 = lässt schwache 情報源 を通じて | > 0.70 = endlose Such-Schleifen (Std: 0.45). Max Rewrites: Std 2.

### 97. 2-段階 Core-Memory の後に MemGPT/Letta-Vorbild (backend/core_memory_manager.py)

**問題:** Wichtige Nutzerpräferenzen と Projektziele gehen verloren, sobald sie aus kurzlebigen Chat-窓 herausfallen.

**Mechanik:** Verankert editierbare Pinned-Working-Memory-Blöcke (<CORE_WORKING_MEMORY>) dauerhaft im システム-プロンプト, getrennt からの großen 書庫-記憶.

**Nutzen:** Garantiert 一つの permanentes, からの エージェント selbst pflegbares Kerngedächtnis について beliebig viele 会議 hinweg.

**価格 & Schattenseite (Trade-off):** 各 angepinnte Block belegt に際して absolut jedem LLM-Aufruf dauerhaft プロンプト-Tokens (10 Blöcke × 800 文字 ≈ 2.000 Tokens Grundlast pro 通知).

**Wann ausschalten? & Schwellenwerte:** Ausschalten 又は Block-Anzahl auf 3–5 reduzieren, wenn kleine Modelle と 共に engem Kontextfenster 又は knappe Token-Budgets genutzt werden. | Schwellenwerte: Max Blöcke: > 15 = hoher Token-Grundverbrauch (Std: 10). Max 文字/Block: < 300 = schneidet 規則 ab | > 1500 = verstopft プロンプト (Std: 800).

### 98. Automatisches Citation- & 情報源-Pinning (backend/citation_pinner.py)

**問題:** Ohne direkte Quellenverweise は のために 利用者 ない überprüfbar, aus welchem Dokument 一つの Faktenaussage の エージェント stammt.

**Mechanik:** Ordnet 文 の 応答 経由 Token-Overlap abgerufenen Quell-Dokumenten zu, fügt Inline-証憑 ([Quelle: ID]) 一つの と prüft Mindest-Belegquote.

**Nutzen:** Macht 各 Aussage revisionssicher nachvollziehbar と deckt unbelegte Behauptungen sofort auf.

**価格 & Schattenseite (Trade-off):** Erhöht Output-Token-Verbrauch um ~10–15 %; 一つの zu hohe Mindest-Abdeckung (> 80 %) führt に際して kreativen 又は zusammenfassenden Fragen zu Fehlwarnungen.

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して freiem Brainstorming, Code-Generierung 又は lockerem Alltags-Chat 無しに Dokumentenbezug. | Schwellenwerte: Min. Abdeckung: < 30% = unbelegte Aussagen | > 85% = Fehlwarnungen に際して Zusammenfassungen (Std: 60%).

## 15. 信頼性の高い実行と決定性（耐久実行）

**99. Structured Outputs & JSON-Schema-保証 (backend/structured_output_enforcer.py)**

**問題:** Sprachmodelle 納品する manchmal ungültiges JSON (Markdown-枠組, fehlende Klammern 又は falsche Datentypen), wodurch nachgelagerte Code-Pipelines abstürzen.

**Mechanik:** Repariert Syntaxfehler im Roh-String 自動の と erzwingt 全ての Pflichtfelder と Datentypen の übergebenen 目標-Schemas.

**Nutzen:** Garantiert 100 % maschinenlesbare, typensichere JSON-Payloads エージェント間で と Backend-接続口.

**価格 & Schattenseite (Trade-off):** 各 Auto-Reparatur-Schleife kostet +1 bis +3 Sek. 遅延 と zusätzliche Tokens. Strikte Schema-実証 lehnt Antworten も のために harmloser Zusatzfelder ab.

**Wann ausschalten? & Schwellenwerte:** Strikte 実証 ausschalten, wenn API-Endpunkte 又は Skripte も と共に flexiblen 又は teilweisen JSON-Feldern umgehen können. | Schwellenwerte: Schema-監査: Strikt vs. Tolerant (Std: Strikt). Max Reparaturen: 1–2 = repariert 98 % aller Tippfehler | > 3 = verschwendet Tokens (Std: 2).

**100. Durable Execution Checkpointing / Pause & Resume (backend/durable_checkpoint_manager.py)**

**問題:** Bricht 一つの 10-schrittiger Recherche- 又は Build-注文 に際して 段階 8 を通じて 一つの Neustart ab, mussten bisher 全ての 7 Vorarbeiten teuer wiederholt werden.

**Mechanik:** Sichert の後に jedem abgeschlossenen 段階完全 en Variablen- と Fortschrittszustand transaktional in SQLite.

**Nutzen:** Unterbrochene Langläufer setzen exakt am letzten gültigen Checkpoint 再び auf, 無しに Arbeit 又は Tokens doppelt zu verbrauchen.

**価格 & Schattenseite (Trade-off):** Checkpointing に際して jedem einzelnen 段階 (N = 1) erzeugt spürbare SQLite-Schreiblast. Auto-Resume に際して Booten できる veraltete Tasks からの Vortag ungewollt 再び anfeuern.

**Wann ausschalten? & Schwellenwerte:** Auto-Resume ausschalten, wenn unterbrochene 任務 の後に 一つの Neustart erst 手動の geprüft と freigegeben となる sollen. | Schwellenwerte: 全て N 段階: 1 = absolut absturzsicher, mehr Disk-I/O | 3–5 = schont SSD (Std: 1). 保管: Std 14 Tage.

**101. Idempotency-鍵 & Tool-Call-Deduplizierung (backend/idempotency_guard.py)**

**問題:** Bei 通信網-Retries 又は エージェント-Schleifen besteht Gefahr, dass dieselbe 電子郵便, 請求書 又は Webhook-Aktion mehrfach ausgelöst wird.

**Mechanik:** Vergibt のために 各 mutierenden Tool-Call 一つの SHA-256-Idempotency-Key im SQLite-Ledger と blockiert identische Folgeaufrufe 以内に の TTL.

**Nutzen:** Verhindert doppelte Außenwirkungen 信頼できる の後に Vorbild moderner Zahlungs-Gateways (Stripe-Prinzip).

**価格 & Schattenseite (Trade-off):** Wenn の 利用者 exakt denselben Befehl 以内に の Sperrfrist (標準 24 h) absichtlich 一つの zweites Mal 実行する will, blockiert の Guard.

**Wann ausschalten? & Schwellenwerte:** TTL-Stunden auf 1 h senken 又は kurzzeitig ausschalten, wenn identische Aktionen im Testbetrieb mehrfach hintereinander ausgeführt となる müssen. | Schwellenwerte: TTL (Stunden): < 1h = kurzer Duplikatschutz | > 48h = blockiert gewollte Wiederholungen (Std: 24h).

**102. Dry-Run / Shadow-Mode のために エージェント-注文 (backend/dry_run_simulator.py)**

**問題:** Neue 又は komplexe エージェント-注文 möchte man の前に の scharfen 実行 oft erst 無しに 危険 an echten Produktionsdaten testen.

**Mechanik:** Fängt im Shadow-Mode 全ての schreibenden 綴-, データベース- と API-Befehle ab と protokolliert stattdessen 一つの detaillierten Mutations-Vorschauplan.

**Nutzen:** Ermöglicht gefahrlose Trockenübungen と 危険-Audits の前に の endgültigen 承認.

**価格 & Schattenseite (Trade-off):** SOLANGE AKTIV, WERDEN KEINE ECHTEN DATEIEN GESCHRIEBEN UND KEINE ECHTEN APIS AUSGEFÜHRT! Vergisst man Schalter, wirkt システム の後に außen hin 'tot'.

**Wann ausschalten? & Schwellenwerte:** Im normalen Produktivbetrieb zwingend AUSGESCHALTET lassen! Nur gezielt のために Trockenübungen (Dry-Runs) と 安全 Audits einschalten. | Schwellenwerte: Hauptschalter standardmäßig AUS (false). Nur のために isolierte Testläufe 有効化する.

**103. Dead-Letter-Queue (DLQ) と共に Auto-再演-Manager (backend/dead_letter_queue.py)**

**問題:** Fehlgeschlagene Webhooks 又は asynchrone Hintergrund-Jobs gehen に際して kurzzeitigen Drittanbieter-Ausfällen 無しに Auffangbecken spurlos verloren.

**Mechanik:** Speichert 各 gescheiterten Job samt 原本-Payload と Fehlergrund in の DLQ-表 と wiederholt ihn と共に exponentiellem Backoff.

**Nutzen:** Kein 注文 geht verloren; fehlgeschlagene Tasks lassen sich jederzeit einzeln 又は gesammelt erneut abspielen.

**価格 & Schattenseite (Trade-off):** Bei dauerhaft fehlerhaften Aufträgen (z. B. ungültiger API-Key) verbrauchen Wiederholungsversuche unnötig 資源; zeitversetzte Replays できる Aktionen verspätet auslösen.

**Wann ausschalten? & Schwellenwerte:** Auto-再演 ausschalten, wenn fehlgeschlagene 注文 ausschließlich 手動の gesichtet と einzeln freigegeben となる sollen. | Schwellenwerte: Max Retries: 1 = bricht に際して kurzem Ausfall ab | > 5 = staut tote Jobs an (Std: 3). Backoff-Basis: Std 10s (10s -> 20s -> 40s).

## 16. セキュリティ、データ保護およびエンタープライズガードレール

**104. Bidirektionaler PII-Scrubber & Re-Hydration (backend/pii_scrubber.py)**

**問題:** Werden 電子郵便 s, Telefonnummern 又は IBANs im Klartext an 外部の Cloud-LLMs übertragen, drohen DSGVO-Verstöße と ungewollter Datenabfluss.

**Mechanik:** Ersetzt sensible Personendaten の前に API-Aufruf 地域 を通じて Platzhalter ([EMAIL_1], [IBAN_1]) と tauscht この in の eintreffenden 応答 再び に対して echten 値 aus.

**Nutzen:** Cloud-Modelle sehen 決してない echte Personendaten, の間に の 利用者 の前に Ort dennoch 完全な Dokumente と Antworten erhält.

**価格 & Schattenseite (Trade-off):** Wenn LLM Platzhalter im Antworttext leicht abwandelt, schlägt Rückübersetzung (Rehydrierung) fehl. Zudem できる Regex-Maskierung manchmal Code-Strings verfälschen.

**Wann ausschalten? & Schwellenwerte:** Ausschalten, wenn ausschließlich 地域の Modelle (Ollama) genutzt となる 又は LLM zwingend echten 電子郵便-Domains/Namen 分析する muss. | Schwellenwerte: Granulare Schalter のために 電子郵便 s, Telefonnummern, IBANs と automatische Rehydrierung (alle standardmäßig 有効な).

**105. Egress-Firewall & Domain-Allowlisting のために エージェント (backend/egress_firewall.py)**

**問題:** Über versteckte プロンプト-Injections in Webseiten könnten エージェント angewiesen werden, 内部の Daten 経由 HTTP-Request an fremde Angreifer-サーバー zu 送信する.

**Mechanik:** Prüft 各 ausgehende 目標-URL に対して konfigurierte Domain-Allowlist と blockiert unautorisierte Hosts so どのように rohe IPv4-住所.

**Nutzen:** Unterbindet Daten-Exfiltration auf Netzwerkebene selbst に際して manipulierten 入力.

**価格 & Schattenseite (Trade-off):** Solange allowlist_only 有効な ist, となる 各 neue Webseite, API 又は 文書化, ない explizit in の Domain-一覧 steht, に際して Web-Recherchen hart blockiert.

**Wann ausschalten? & Schwellenwerte:** Ausschalten (oder benötigte Domains ergänzen), wenn の Research-Agent frei im offenen Web recherchieren soll. | Schwellenwerte: Erlaubte Domains: Kommagetrennte Whitelist. Roh-IP-Blockade schützt の前に direkten IP-Exfiltrationen.

**106. Granulare エージェント-RBAC & 綴-Scope-Jails (backend/agent_rbac_jail.py)**

**問題:** Wenn 各 Sub-Agent vollen Schreibzugriff auf gesamte Projektverzeichnis besitzt, できる 一つの Frontend-Agent versehentlich Backend-Kernel-綴 又は .env-Schlüssel überschreiben.

**Mechanik:** Bindet 各 Agentenrolle an 一つの feste Werkzeugliste と 一つの erlaubtes 目録-Jail (z. B. static/ のために Frontend-エージェント).

**Nutzen:** Setzt Prinzip minimaler 権利 (Least Privilege) のために 各 einzelnen 過程 im Betriebssystem durch.

**価格 & Schattenseite (Trade-off):** Verhindert, dass 一つの 管理者-Agent legitim Konfigurationsdateien 以外に の Projektordners wartet 又は .env-雛形 direkt überprüft.

**Wann ausschalten? & Schwellenwerte:** Nur のために dedizierte Systemwartungs-任務 kurzzeitig lockern; の .env-Schutz sollte im Normalbetrieb 常に 有効な bleiben. | Schwellenwerte: Schalter のために Path-Jail, .env-Sperre と Systemordner-Sperre (/etc, /root, /proc).

**107. Deterministischer Output-Linter & Policy-Checker (backend/output_policy_linter.py)**

**問題:** Sprachmodelle verfallen gelegentlich にもかかわらず システム-プロンプト in künstliche KI-Floskeln 又は generieren riskante Shell-Snippets.

**Mechanik:** Scannt generierten 本文 in Echtzeit mittels regulärer Ausdrücke, entfernt Sycophancy-Opener/AI-Slop と entschärft destruktive Befehle.

**Nutzen:** Sichert sprachliche Präzision と technische セキュリティ 無しに zusätzlichen LLM-Prüfaufruf.

**価格 & Schattenseite (Trade-off):** Destruktiv-濾過 できる も harmlose Aufräum-Skripte 又は gewollte データベース-Migrationen blockieren, wenn darin Befehle どのように DROP 又は rm -rf vorkommen.

**Wann ausschalten? & Schwellenwerte:** Destruktiv-阻害 kurzzeitig ausschalten, wenn gezielt サーバー-Aufräumskripte 又は SQL-Reset-Migrationen erstellt werden. | Schwellenwerte: Schalter のために KI-Floskeln 除去する, Destruktive Befehle blockieren と Auto-Bereinigung.

**108. Manipulationssichere Execution Receipts / SHA-256-Quittungen (backend/execution_receipt_ledger.py)**

**問題:** Für 遵守-Audits (EU AI Act / ISO 42001) reichen einfache 本文-記録 ない aus, なぜなら sie nachträglich verändert となる können.

**Mechanik:** Verkettet 各 Task-清算 (模型, Tokens, USD-費用, ausgeführte Tools) と共に SHA-256-Hash の Vorgängereintrags in 一つの Blockchain-ähnlichen Kette.

**Nutzen:** 各 nachträgliche Manipulation an 費用- 又は Entscheidungsprotokollen fällt に際して の Integritätsprüfung sofort auf.

**価格 & Schattenseite (Trade-off):** Verursacht minimalen Rechen- と 記憶-Overhead (~2 ms と ~2 KB SQLite-記憶 pro Task); Hash-Kette wächst について Monate kontinuierlich an.

**Wann ausschalten? & Schwellenwerte:** Kann im Produktivbetrieb dauerhaft aktiviert bleiben; のみ auf extrem speicherlimitierten Kleinstsystemen ausschalten. | Schwellenwerte: Hash-Verkettung (SHA-256) + detaillierte 費用-Aufschlüsselung pro Lauf.

## 17. 自己最適化、評価および群知能

**109. Automatischer プロンプト-Optimizer の後に DSPy/TextGrad-Prinzip (backend/prompt_optimizer.py)**

**問題:** Statische システム-Prompts wiederholen dieselben 障害 常に wieder, solange 無 Mensch 手動の nachbessert.

**Mechanik:** Wertet reale 障害-記録 一つの 役割 aus, erkennt systematische 弱 と generiert gezielte Guardrail-Zusatzregeln のために システム-プロンプト.

**Nutzen:** エージェント lernen kontinuierlich aus gescheiterten Läufen と härten ihre Instruktionen selbstständig.

**価格 & Schattenseite (Trade-off):** は Auto-Apply 有効な, できる 一つの fehlerhaft abgeleitete 規則 行動 の エージェント dauerhaft verzerren (プロンプト-Drift) と システム-プロンプト aufblähen.

**Wann ausschalten? & Schwellenwerte:** Auto-Apply im Produktivbetrieb ausgeschaltet lassen, damit vorgeschlagene プロンプト-Patches erst の後に menschlicher 監査 übernommen werden. | Schwellenwerte: Ab Fehlern: 1 = überreagiert auf Einzelfehler | > 8 = lernt sehr träge (Std: 3). Max 規則: > 10 = bläht システム-プロンプト auf (Std: 5).

**110. Gold-標準 Regression-試験-Suite / Eval-走者 (backend/eval_regression_runner.py)**

**問題:** Eine vermeintlich kleine Änderung am システム-プロンプト 又は 模型 できる bestehende 安全 又は Formatregeln unbemerkt verschlechtern.

**Mechanik:** Führt の前に の 買収 neuer 設定 automatisierte Gold-標準-試験事例 aus と prüft Bestehensquote so どのように verbotene Muster.

**Nutzen:** Verhindert Qualitäts-Regressionen im Produktivbetrieb を通じて 一つの objektives Quality-Gate.

**価格 & Schattenseite (Trade-off):** 各完全 e Eval-通流 feuert 全ての hinterlegten 試験-Prompts に対して LLM-API (kostet je の後に Testumfang ~5.000–25.000 Tokens と 15–60 Sek. 存続期間).

**Wann ausschalten? & Schwellenwerte:** Deploy-Blockade ausschalten, wenn schnelle experimentelle プロンプト-試験 無しに 待機時刻 durchgeführt werden. | Schwellenwerte: Min. Pass-Rate: < 75% = lässt Verschlechterungen を通じて | 100% = blockiert 既に に際して kleinsten 乖離 (Std: 90%).

**111. Multi-Armed Bandit 模型-Router (backend/bandit_model_router.py)**

**問題:** Feste 模型-Zuweisungen reagieren ない darauf, wenn 一つの Cloud-提供者 plötzlich langsamer となる 又は erhöhte Fehlerraten zeigt.

**Mechanik:** Bepunktet 各 模型 laufend の後に realer Erfolgsquote (60 %) と Geschwindigkeit (40 %) と wählt dynamisch besten Kandidaten.

**Nutzen:** Maximiert Antwortqualität と Tempo 自動の に基づき echter 存続期間-遠隔測定.

**価格 & Schattenseite (Trade-off):** Durch Explorationsrate (Epsilon, z. B. 0.15 = 15 %) testet の Router absichtlich 常に 再び alternative Modelle aus, im Einzelfall langsamer sein können.

**Wann ausschalten? & Schwellenwerte:** Ausschalten (oder Epsilon auf 0.00 setzen), wenn のために 各 Lauf 一つの 100 % festes, vorhersagbares 模型 erzwungen となる soll. | Schwellenwerte: Epsilon: 0.00 = nutzt stur beste 模型 | > 0.30 = probiert zu oft schwächere Modelle (Std: 0.15). Erfolgs-/遅延-Gewicht: Std 0.60 / 0.40.

**112. Dynamisches 暫定的な-班-Spawning (backend/ephemeral_team_spawner.py)**

**問題:** Ein einzelner Generalisten-Agent は に際して großen Querschnittsaufgaben (アーキテクチャ + UI + Security + QA) 迅速な überfordert.

**Mechanik:** Rekrutiert passend への Missions-Briefing 一つの temporäre 専門家-Task-Force と löst sie の後に erledigter Mission 再び auf.

**Nutzen:** Bündelt Spezialwissen bedarfsgerecht 無しに dauerhaften 資源-Overhead.

**価格 & Schattenseite (Trade-off):** Ein 4-köpfiges Ad-hoc-班 vervielfacht Token-Verbrauch (3x bis 5x) と benötigt zusätzlichen Koordinations-Overhead の間に Teammitgliedern.

**Wann ausschalten? & Schwellenwerte:** Ausschalten に際して einfachen Einzelaufgaben 又は knappem Token-予算; Max. Teamgröße im Alltag auf 3–4 begrenzen. | Schwellenwerte: Max Teamgröße: 2–4 = ideal | > 5 = hoher Kommunikations-Overhead & 5x Token-Verbrauch (Std: 4).

**113. 実況-資源-Protector & Host-Backpressure (backend/resource_backpressure.py)**

**問題:** Parallele 容器, Browser-Audits と 地域の 過程 できる に際して Lastspitzen Host-RAM 又は CPU überlasten (OOM-Kill).

**Mechanik:** Überwacht CPU-, RAM- と Disk-Metriken in Echtzeit と drosselt 低い priorisierte Hintergrund-Tasks ab 85 % 稼働率 自動の.

**Nutzen:** Schützt サーバー 信頼できる の前に Abstürzen を通じて Überlastung と hält kritische Kernprozesse jederzeit reaktionsfähig.

**価格 & Schattenseite (Trade-off):** Sind Schwellenwerte zu 低い eingestellt (CPU < 70 % 又は RAM < 70 %), となる Hintergrund-任務 既に に際して normalen kurzen Lastspitzen ständig ausgebremst.

**Wann ausschalten? & Schwellenwerte:** Nicht 完全 ausschalten — stattdessen に際して starker 機材 CPU-/RAM-Schwellen auf 90 % anheben. | Schwellenwerte: Max CPU/RAM: < 70% = drosselt に際して normaler Arbeit | > 92% = OOM-Gefahr (Std: 85%). Min. freie Disk: Std 2.0 GB.

## 18. 柱 3：経済・ERP（アルファ）およびテナントスイート

| モジュール / View | Funktionsumfang & Technische Merkmale (v0.5.1) |
|---|---|
| **住所 & CRM + PocketBase-Sync** | Zentraler Kontakt-, 売掛金- (10000+) と Kreditorenstamm (70000+), 360°-連絡先引出, Multi-Channel-Timeline, 暦/再提出 so どのように LinkedIn- & 媒体-Sync について PocketBase (Ports 3005/8090/8092). |
| **品目 & 役務 (ART)** | Stammdatenkatalog ab ART-1001 のために Dienstleistungen, Stundensätze, 定額料金, SaaS-定期購読 so どのように physische 製品 と共に Lagerbestandsführung. |
| **案件 & 契約** | Durchgängige Belegkette 見積 (ANG-YYYY-NNNN) → 注文 (AB-YYYY-NNNN) 含む gewichteter 案件導管, 売上予測 と 契約期限電探 (30/14/7-Tage-Alarme). |
| **会計 & DATEV** | Ausgangs- と Eingangsbelege の後に § 14 UStG, GiroCode-Zahlungs-QR, Finanz-分析, automatisches 督促事務 (Dunning Engine) と EXTF/CSV-搬出 のために DATEV. |
| **法人 & テナント-Isolation** | Mehrere eigene Unternehmungen (company_id) と共に physisch getrennten SQLite-データベース im WAL-Modus, eigenen Nummernkreisen, Briefbögen と 雛形-Editor. |

## 19. アーキテクチャ・セキュリティ・デプロイメント機能 (v0.5.0 – v0.5.1)

| Feature / Subsystem | 具現化 & Betriebsnutzen (v0.5.1) |
|---|---|
| **Modulare Router-アーキテクチャ** | Auslagerung in dedizierte FastAPI-Router の下に archon/routers/ (main.py の 16.254 auf 1.875 行 reduziert, -88 %) と zentrale Konfigurationsschicht archon/core/config.py. |
| **Security Hardening (Archon Hardened)** | Schließung の 25 kritischen と 68 hohen 監査-Befunden; Starlette Threadpool-Entlastung, gehärtete UI-DOM-Pfade と Klasse-A Docker-Admin-Proxy のために hermes_updater. |
| **RBAC-Benutzerverwaltung (17.10)** | Granulare 役割 (admin, operator, viewer) について backend/auth_service.py と共に 100.000 Runden PBKDF2-HMAC-SHA256 合言葉-Hashing と Single-User/Multi-User-Modus. |
| **2-段階 システム更新 (17.11)** | Sofortige 表示 lokaler 版/HEAD via ?local_only=true (~0,25 s) plus asynchroner GitHub-Abgleich, Pre-更新 SQLite Online-Snapshots と 1-Klick-Rollback. |
| **AnythingLLM Proxy & Chroma Stack** | Nahtlose Einbettung の AnythingLLM in 中央図書館 について /anythingllm-proxy 含む /assets/* Fast-Path と lokalem ChromaDB Vektor-索引. |
| **Portabler 1-Click-Installer** | Plattformübergreifende Erstinstallation について setup.bat, setup.ps1 と install.sh と 共に docker-compose.yml, .env.example, INSTALL_AGENT.md と 9-sprachiger GitHub-Doku. |
| **四眼 品質関門 & Code-Sandbox** | Automatische Zweitprüfung im Kanban の前に Ticket-締結 so どのように 対話型の Code-Sandbox im Showroom (4.4.1) への 実況-Vorschau erzeugter Artefakte. |

# 20. 五十音順用語集・索引

主要モジュール、アーキテクチャ構成要素および機能の五十音順用語集 (v0.5.1) 正確なページ番号付き.

| Feature / 用語 | Kurzbeschreibung | 頁(n) |
|---|---|---|
| **3-Tier 費用-Routing** | Automatische Modellauswahl in LOW / MEDIUM / HIGH の後に Aufgabenkomplexität と 予算. | — |
| **Anti-AI-Slop & STE100** | Stil- と Wortfilter への Vermeidung unnötiger Floskeln と Einsparung の Output-Tokens. | — |
| **AnythingLLM Proxy** | Lokale Einbettung の AnythingLLM + ChromaDB について /anythingllm-proxy in の 中央図書館. | p. 3, 32 |
| **Approval Gate (HITL)** | Menschlicher Prüfschritt の前に externen 電子郵便 s, 記帳 又は kritischen Dateizugriffen. | p. 3 |
| **Archon Hardened (v0.5.1)** | 安全 と Stabilitäts-放出 と共に modularer Router-Struktur と geschlossenen 監査-Befunden. | p. 3, 32 |
| **Bandit 模型-Router** | Adaptiver Router (Multi-Armed Bandit) への dynamischen Modellwahl の後に Trefferquote と 費用. | p. 31 |
| **Bot 画面 (Web-案内者)** | 視覚の Ausführungsmonitor のために Browser-Interaktionen via Playwright. | — |
| **ChromaDB** | Lokale Vektordatenbank のために semantisches Dokumenten-Retrieval in 柱 2. | p. 32 |
| **Context-Compaction & LRU-K** | Speicherauslagerung と Komprimierung langer Chat-Kontexte に際して Erreichen の RAM-Limits. | — |
| **Corrective RAG (CRAG)** | Selbstprüfende Wissensabfrage と共に Relevanzbewertung の前に Antwortgenerierung. | p. 27 |
| **DAG-Scheduler** | Zweistufige Ausführungsplanung abhängiger Teilaufgaben と共に parallelem Fan-Out / Fan-In. | p. 13 |
| **DATEV-搬出 & GiroCode** | EXTF/CSV-接続口 のために Buchhaltungsdaten と EPC-Zahlungs-QR auf 請求書. | — |
| **Dead-Letter-Queue (DLQ)** | Fehlerwarteschlange と共に automatischer Wiederholungssteuerung のために gescheiterte Jobs. | p. 29 |
| **Decision Engine (55 フレームワーク)** | Strategiemodul のために strukturierte Boardroom-討論 について 9 方法論-Kategorien. | — |
| **DSPy / TextGrad Optimizer** | Automatische Verbesserung の システム-Prompts に基づき realer Evaluations-Metriken. | — |
| **Durable Execution & Time-Travel** | Unterbrechbare Langläufe と共に SQLite-Checkpoints と schrittweisem Execution-再演. | — |
| **Egress-Firewall** | Ausgehende 通信網- と Domain-統制 のために エージェント-Toolaufrufe. | p. 29 |
| **ERP-Belegkette (ANG -> AB -> RE)** | Verknüpfte Vertriebs- と Buchhaltungsdokumente 含む Pipeline-予測 と 督促事務. | — |
| **Evey / Hermes 差込** | Modulare システム-拡張 のために 記憶, 委任, Verifikation と 接続口. | — |

| Feature / 用語 | Kurzbeschreibung | 頁(n) |
|---|---|---|
| **Hermes Master Orchestrator** | Zentrale Kern-Engine のために Zielzerlegung, 委任 と 統合 im Daemon auf Port 8420. | p. 11 |
| **Hybrid-Search (FTS5 + RRF)** | Kombination aus BM25-Volltextsuche と Vektorsuche 経由 Reciprocal Rank 合併. | p. 26 |
| **Idempotency-鍵 & Dry-Run** | Schutz の前に doppelter Tool-実行 so どのように Simulationsmodus 無しに Schreibzugriff. | — |
| **JEV Router (0-Token Intent)** | Deterministischer Vorfilter 無しに KI-費用 のために eindeutige Kurzbefehle. | p. 10 |
| **Knowledge-Graph** | Lokaler Tripel-記憶 のために strukturierte Entitätsbeziehungen neben Vektordaten. | p. 26 |
| **MCP (Model Context Protocol)** | Standardisierte Treiberarchitektur への sicheren Werkzeuganbindung. | p. 6, 14 |
| **OmniRoute AI 門** | Multi-提供者-門 含む Free-Tier-Routing, Latency-Hedging と Eval-競技場. | — |
| **PBKDF2 / Argon2id RBAC** | 役割- と 合言葉-Absicherung im System Control Center (Tab 17.10). | — |
| **PII-Scrubber & Re-Hydration** | Bidirektionale Anonymisierung sensibler Personendaten の前に externen LLM-Calls. | p. 29 |
| **Pre-Flight Kostenschätzer** | Vorabberechnung erwarteter Token-費用 含む Task-予算-Stopp に際して Überschreitung. | — |
| **Research Lab** | Mehrstufige Web- と Quellenrecherche と共に Evidenz-Triangulation. | p. 3, 7, 21 |
| **Router-アーキテクチャ (archon/routers)** | Modulare Backend-Aufteilung in schlanke FastAPI-Router (-88 % Dateigröße in main.py). | p. 32 |
| **Semantic Caching** | 24h-Zwischenspeicherung inhaltlich ähnlicher Prompts への Token-Einsparung. | — |
| **SLA-契約 (契約執行)** | Formale 引渡- と Qualitätszusicherungen の間に spezialisierten エージェント. | p. 25 |
| **SQLite WAL Connection-Pool** | Schreibsicherer Multi-Thread-Datenbankzugriff について Write-Ahead Logging. | p. 26 |
| **System Control Center (11 Tabs)** | Zentrale Administrationskonsole 含む 利用者/RBAC と 2-段階 システム更新. | p. 8, 25 |
| **Token Guard & Loop-Killer** | Echtzeit-Schutz に対して Endlosschleifen (Jaccard-監査) と Token-Überschreitungen. | — |
| **事前葬儀 & 合意関門** | Simulierter 障害-圧力試験 と Mehrfach-エージェント-投票 の前に kritischen 承認. | — |
| **事象網 & Webhook-Trigger** | Ereignisgesteuerte 処理 eingehender Webhooks と Ordneränderungen. | — |
| **四眼 品質関門** | Unabhängige 監査 abgeschlossener Kanban-任務 を通じて 一つの 監査者-エージェント. | p. 3, 14, 32 |
| **暫定的な Code-Sandbox** | 揮発性の, isolierte 容器-存続期間 のために sicheren Python- と Skriptcode. | — |
| **法人 & Mandantenfähigkeit** | Physische データベース-Trennung mehrerer Gesellschaften 経由 company_id. | — |

| Feature / 用語 | Kurzbeschreibung | 頁(n) |
|---|---|---|
| **監査-Log (SHA-256 Hashkette)** | Revisionssichere Protokollkette aller システム- と Agentenaktionen. | p. 18 |
| **監視役 & Deadlock-Supervisor** | 30-Sekunden-監視 hängender Tasks, 容器 と automatischer Selbstheilung. | — |
| **睡眠周期 統合** | 夜間の Deduplizierung と Langzeit-Verdichtung の エージェント-Gedächtnisses. | — |